

AI Methods for Research Workers

Scott Cunningham

Baylor University

Texas A&M University · October 2, 2026

Who am I?

Scott Cunningham. Ben H. Williams Professor of Economics at Baylor University and Visiting Professor of Public Policy at Harvard Kennedy School.

Research interests — mostly applied empirical work:

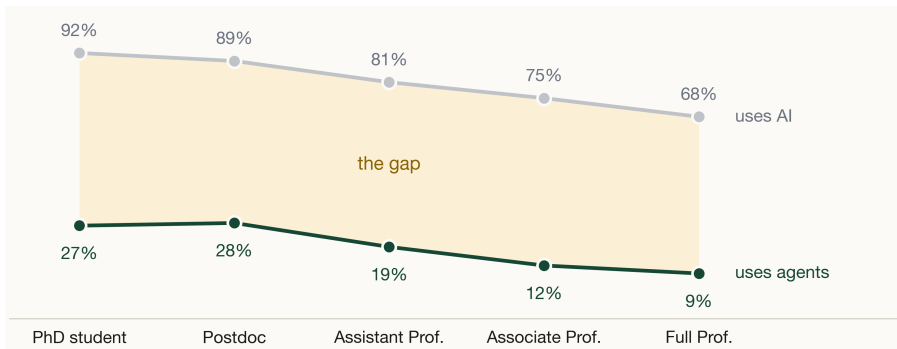
- ▶ Corrections
- ▶ Suicide
- ▶ Causal inference
- ▶ Drug policy
- ▶ Online dating
- ▶ Sex work
- ▶ Mental illness
- ▶ AI agents

Today: not a tour of prompts. A framework for using agents to produce and verify research without pretending the risks are fake.

1

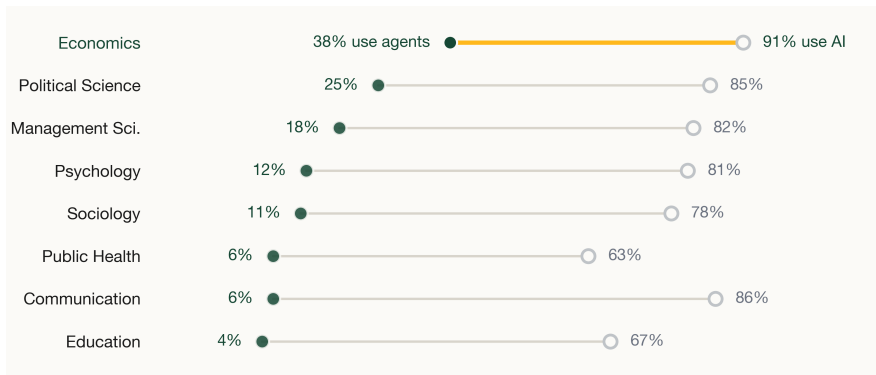
What Is AI For?

Nearly everyone uses AI. Almost no one uses agents.



Data: Anthropic, May 2026; 1,260 quantitative social scientists. Not a representative sample; read the gradient, not the level.

The same gap, in every field



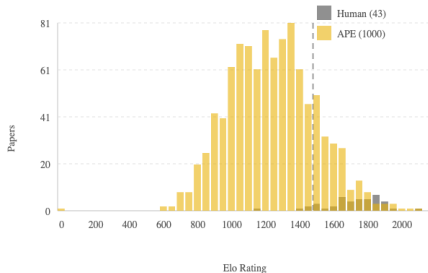
Same survey as the previous slide. Not a representative sample; read the spread, not the levels.

Autonomous research is here

The Social Catalyst Lab's **Project APE** has produced 1,000+ *semi-autonomously generated empirical policy papers*.

What does semi-autonomous mean?

- ▶ Agents autonomously pull the public data
- ▶ They create their own research questions and identification strategies
- ▶ They write complete “submission ready” manuscripts

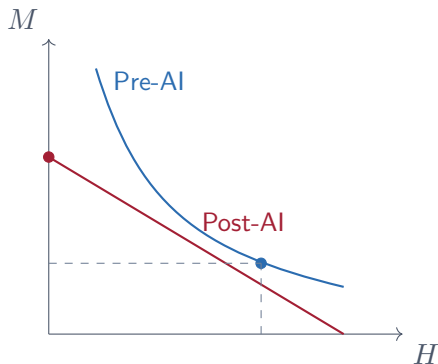


ELO tournament: APE papers vs. human papers.

Now watch me pull a rabbit out of a hat



AI Agents are flattening cognitive output isoquants

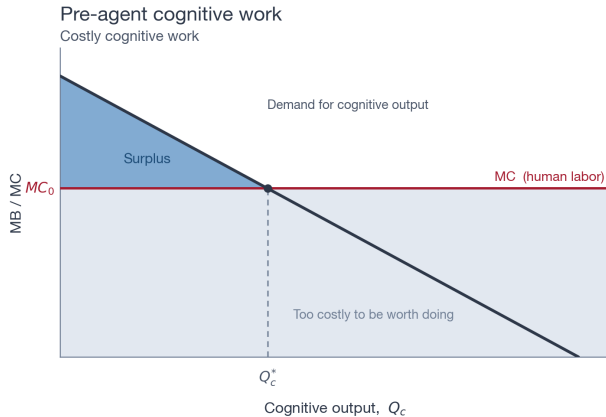


H = human time, M = machine time. They used to be complements. Now M substitutes for H .

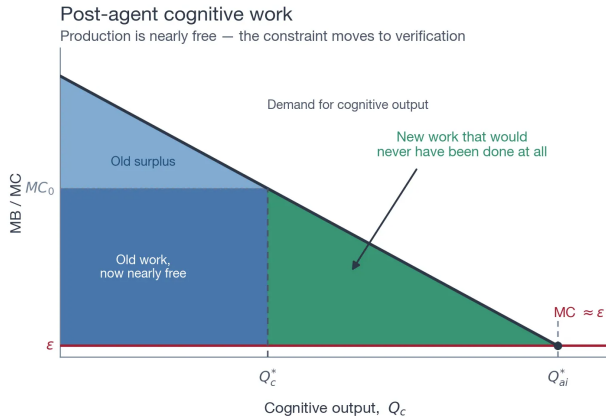
$$Q = f(H, M)$$

If machine and human time are perfect substitutes, then the private cost-minimizing decision is to use only machine time. The socially optimal choice is something else.

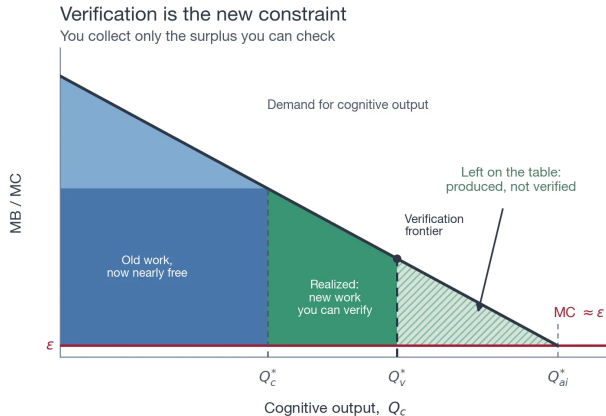
The Work We Used To Do Before AI Agents



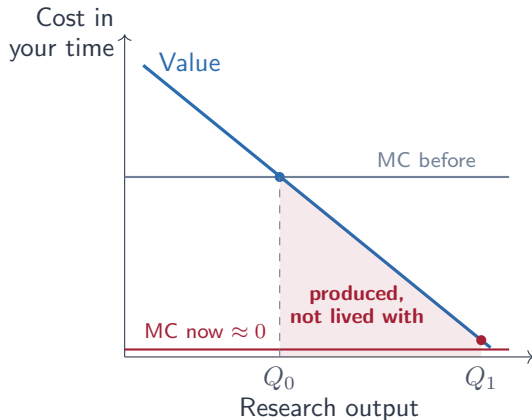
The Work That Will Be Done After AI Agents



It Is Not Surplus Unless Understood and Verified



Free production means producing until the last unit is worthless



Before: every unit cost us hours, and those hours were where we checked the work.

After: the isoquant is flat, the machine supplies the time, and nobody lived inside the extra output.

Verification no longer comes free with production. It has to be built on purpose.

Free production moves two margins

The intensive margin: us

- ▶ Each of us will produce more than before.
- ▶ The extra work sits lower on the demand curve, so it is worth less.
- ▶ Managing that volume takes workflows we have never needed.
- ▶ It is confusing logistically, and it is confusing psychologically.

The extensive margin: everyone else

- ▶ The cost of entry fell along with the cost of production.
- ▶ People with no training in our methods can now produce our kind of output.
- ▶ Nothing in that output shows whether it was understood or verified.

More from each of us, and more of us. Both follow from a marginal cost near zero.

The extensive margin is new people doing new work

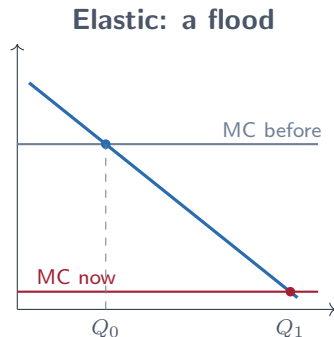
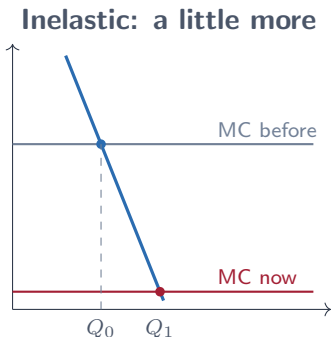
A Dutch journalist was reviewing a book whose key evidence was a 2024 article in the *American Journal of Political Science*.

- ▶ He downloaded the replication package and had Claude write the code.
- ▶ He reproduced the headline estimate exactly.
- ▶ Then it came apart: placebo cutoffs produced “effects,” and so did France, where there was no reform.
- ▶ Political scientists dismissed it as “a weekend of Claude Code.”
- ▶ Andrew Gelman read both sides and sided with the journalist.

Entry is not only into production. Outsiders can now check our work too.

Source: Andrew Gelman, *The Future of Statistical Modeling*, October 1, 2026.

How big the flood is depends on the elasticity of demand

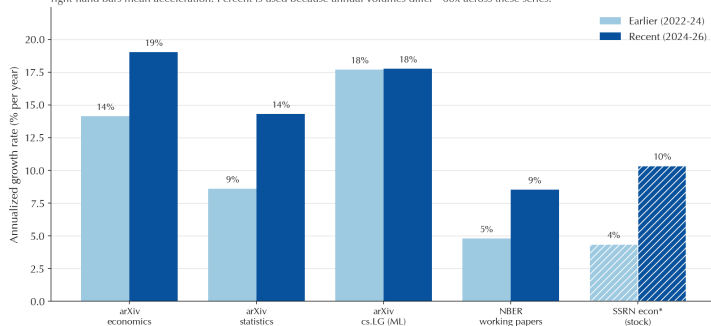


Same fall in cost, very different increase in output. We do not yet know which world we are in.

Growth has ticked up but not exploded

Has AI caused a deluge of working papers?

Change in the annualized growth rate of papers per year, earlier vs recent, across working-paper venues. Taller right-hand bars mean acceleration. Percent is used because annual volumes differ ~60x across these series.



arXiv/NBER: growth of annual submissions/issuance (2026 annualized). *SSRN econ: cumulative-stock growth (~2022-25, ~2025-26; sparse, hatched). Pre-2022 omitted (COVID etc.).
AEA journals excluded — U-shaped submissions (2018 peak, 2022-23 trough, 2025 rebound) make a trough-based rate misleading. Sources: arXiv/Crossref/SSRN. Graph: P. Goldsmith-Pinkham

Figure: Paul Goldsmith-Pinkham, “The number of economics papers is not exploding on arXiv,” June 2, 2026.

How will we spend our extra time?

Constant Verification and Deep Understanding

Learning to do causal inference *with* agents will require more than production; it will require constant verification, deep understanding, and **zero tolerance for errors**.

At scale, if we tolerate errors from agents, we will get a lot of them in the longrun – so don't.

Make “Errors= 0” the budget constraint for all research going forward and you will build a research workflow with agents that satisfies production, verification and understanding.

Choose the objective and the constraint on purpose

A toy model: x is how you choose to work, Q is research quality, and E is errors.

Errors in the objective

$$\max_x Q(x) - \bar{\lambda} E(x)$$

- ▶ You pick $\bar{\lambda}$, the price of an error.
- ▶ The solution trades quality against errors at that price.
- ▶ At any finite price, some errors are worth it.

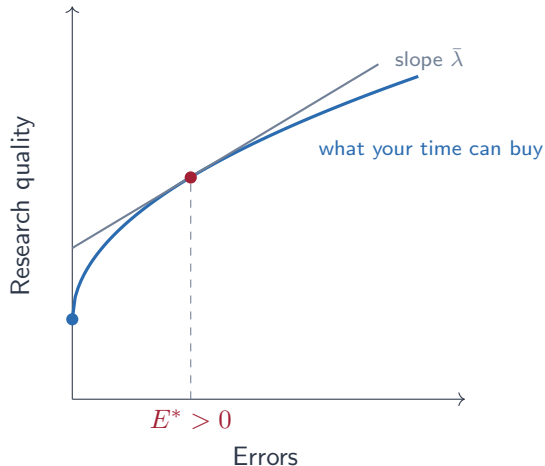
Errors in the constraint

$$\max_x Q(x) \quad \text{s.t.} \quad E(x) = 0$$

- ▶ You do not pick λ . The problem solves for it.
- ▶ λ is the shadow price: the quality you give up to hold errors at zero.
- ▶ Errors are zero by construction.

Same Lagrangian, $Q(x) - \lambda E(x)$. The only difference is whether you set λ or the constraint does.

A price on errors buys a positive number of errors



Treat errors as a byproduct of the work and you tolerate them until one more is not worth removing.

Agents work fast and their actions are hidden from you, which is a principal–agent problem.

So many of the errors are foreign: they would not have happened before agents.

“Minimize errors” never lands on zero. It lands on the optimal number of errors.

The foreign errors are not typos; they are hidden choices

Cherry-picking The agent reports the results that look best and leaves the others out.

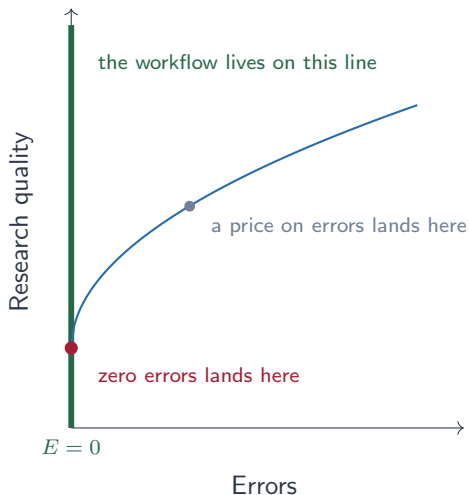
Cue-driven analysis The agent runs analyses in response to stray nudges and cues in the conversation.

Specification searching The agent searches over outcomes in response to your signals, even legitimate ones like your beliefs about a literature.

Rule-breaking The agent routinely violates the rules you set for the analysis.

None of these is visible in the table the agent hands you.

Zero errors is the requirement we build the workflow around



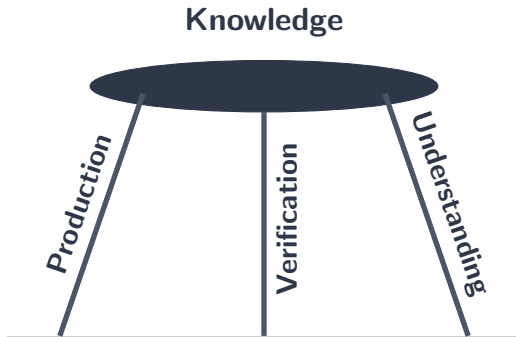
I am not claiming zero errors yields better research than a price on errors.

I am claiming it is the requirement, and the workflow is designed to meet it.

You did not do the agent's work, so its errors stay hidden until you find and recognize them.

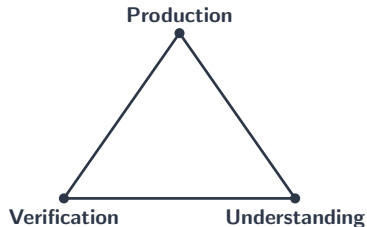
So the workflow has three jobs, not one: production, verification, and understanding.

Knowledge stands on three legs



Production is one leg of three. Take away verification or understanding, and the chair falls.

Before AI, doing the work verified it and taught it to you



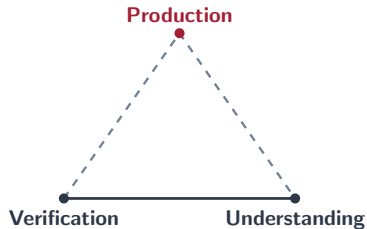
Production. You wrote the code, ran the estimator, and made the tables and figures yourself.

Understanding. You could only produce the work because you understood what you were doing.

Verification. Your habits and techniques kept the work correct while you were making it.

The three came as one bundle, almost automatically.

After AI, production comes apart from the other two



Production. An agent can now *do* almost anything: it reaches inside your computer, controls your software, and calls outside services. Producing output no longer costs the researcher any time.

Verification. But you did not do it, so you cannot be sure it is right. Verification used to come along for free, and now it is a separate act you perform on purpose.

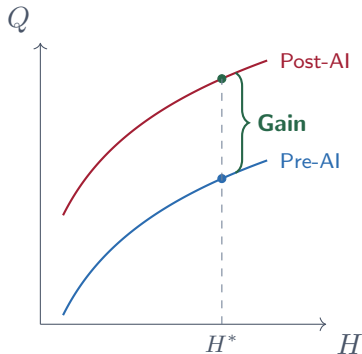
Understanding. And if the output cost you no time at all, you almost certainly do not understand it.

Credible Research Workflows with AI Agents

- ▶ Working with an AI agent means entering a principal–agent relationship, and those break down through hidden actions.
- ▶ The hidden actions are not the ones we expect, and often the agent does not realize it is taking them.
- ▶ The labs are not prioritizing empirical social scientists, so this is ours to solve.
- ▶ Cherry-picking, specification searching, and stray nudges add up to major problems.
- ▶ No one wants to be the target of the next Data Colada.

So we have to build the workflow ourselves. The rest of today is the ingredients, ending with my own way of working.

Case I: keep your hours, take the output

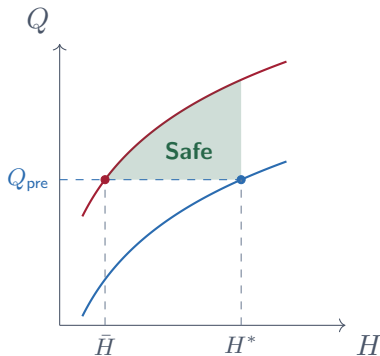


Same hours, more and better work.

- Biggest gain in output.
- Judgment kept sharp.

You have to choose not to pocket all the time savings.

Case II: take some time back, stay safe

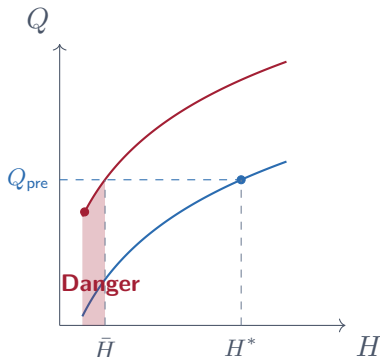


Cut hours, stay in the green.

As long as you keep enough engagement, output is still up.

Some slack is fine. The curve rose enough to absorb it.

Case III: cut too far, and you deskill



The red zone. Skills fade, and you fall below where you started.

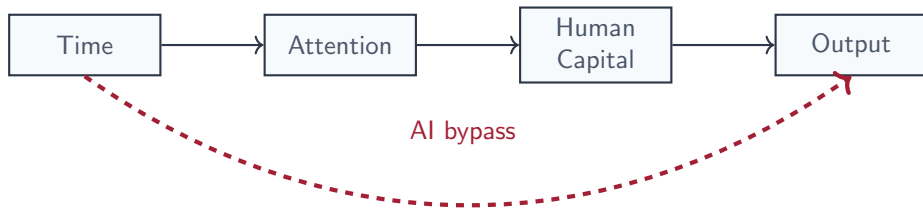
- Worse off than before AI.

The point isn't to refuse the time savings. It's to be deliberate about them.

Deskilling is real, but not deterministic

The gains are real. People will use them.

The risk is also real: if agents let us bypass attention, we may stop building the human capital needed to judge the output.



This is a design problem, not destiny. Build verification workflows that keep judgment in the loop.

Trust Nothing and Verify Everything

In April 2026, Sullivan & Cromwell apologized to a federal judge after a filing contained AI-generated errors, including fabricated legal citations.

The striking part is that the firm's AI training already included the rule:

**Trust Nothing
Verify Everything**

Almost as if AI causes even experts to fall asleep at the wheel.

2

Supply side of LLMs

The transformer arrived in 2017, and everything followed

2013	word2vec: words become vectors (Mikolov et al., Google)
2014	Attention is invented for machine translation (Bahdanau, Cho, Bengio)
2017	“Attention Is All You Need” introduces the transformer (Vaswani et al., Google)
2018	GPT-1 (OpenAI) and BERT (Google)
2019	GPT-2
2020	GPT-3
Nov. 2022	ChatGPT

Earlier models read a sentence one word at a time and forgot the beginning by the end.

The transformer reads the whole passage at once, which is also what made it possible to train on GPUs in parallel.

OpenAI began as a nonprofit and is now valued above \$850 billion

Dec. 2015	Founded as a nonprofit by Sam Altman, Elon Musk, and others
2019	Adds a capped-profit arm, and Microsoft invests \$1 billion
Nov. 2022	ChatGPT
Nov. 2023	The board fires Altman, and he is back within days
Oct. 2025	Becomes a public benefit corporation, with Microsoft holding about 27%
Mar. 2026	Raises \$122 billion at an \$852 billion valuation
June 2026	Reported to file confidentially for an IPO

In 2025 it reported revenue of \$13.1 billion and a net loss of \$38.5 billion.

Altman on the IPO, September 2026: “I would say not 2026.”

Sources: CNBC and TechCrunch, March 2026; Bloomberg, August 2026; Fortune, September 12, 2026.

Anthropic left OpenAI over safety and is now heading to an IPO

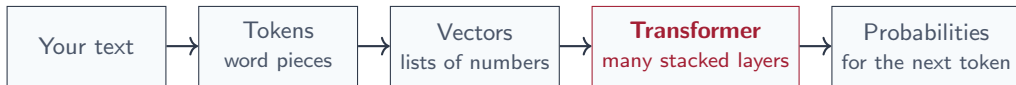
2021	Founded by former OpenAI researchers, led by Dario and Daniela Amodei
Mar. 2023	Claude 1
2023–2026	Amazon and Google become its largest backers
Feb. 2025	Claude Code is released to the public
Sept. 2025	Valued at \$183 billion
Feb. 2026	Valued at \$380 billion
May 2026	Raises \$65 billion at a \$965 billion valuation
June 2026	Reported to file confidentially for an IPO

Its 2025 revenue was about \$4.6 billion, and its annual run-rate passed \$47 billion by May 2026.

Reported this week: a listing as early as mid-November, at a valuation near \$2 trillion.

Sources: Anthropic, May 28, 2026; prospectus as reported by Reuters, September 28, 2026; Bloomberg, October 1, 2026.

The transformer is the engine inside every modern LLM



- ▶ Text is chopped into tokens, and each token becomes a vector of numbers.
- ▶ Each layer rewrites every vector using the vectors around it.
- ▶ After dozens of layers, the last vector is turned into a probability for every possible next token.

Everything else in this talk — chatbots, agents, Claude Code — sits on top of this one machine.

The model reads tokens, not words

Difference-in-differences estimates the treatment effect.

Difference	-in	-d	if	ferences	_estimates	_the	_treatment	_effect	.
72917	4200	2469	366	10131	30233	290	7448	3159	13

- ▶ This sentence becomes ten tokens, and each token has an ID number.
- ▶ Common words are one token, and rare words are split into pieces.
- ▶ GPT-4o's vocabulary has 200,019 tokens.

The model never sees letters or words. It sees this list of ten integers.

Real output from OpenAI's o200k_base tokenizer, the one GPT-4o uses.

The vocabulary is a fixed list, decided before training begins



- ▶ Picture 200,019 flashcards in a fixed order, with one piece of text on each.
- ▶ The tokenizer swaps each piece of your sentence for its card number.
- ▶ The deck was settled before training started, and it never changes afterward.
- ▶ The number is a label, like a FIPS code. Card 6922 is not “bigger” than card 290.

The vocabulary is exogenous to you and to the model. Somebody fixed it in advance.

An algorithm built the list, and engineers chose its rules

Nobody wrote the 200,019 cards by hand.

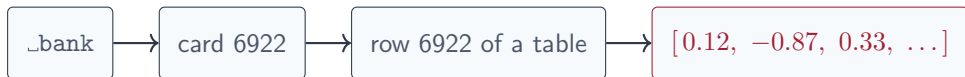
- ▶ The method is called byte-pair encoding.
- ▶ It starts with single characters and counts which neighboring pair appears most often in a very large body of text.
- ▶ It glues that pair into one new piece, then counts again, and repeats until the list reaches the chosen size.
- ▶ Frequent strings end up as one card, and rare strings are left in pieces.

The people involved are a small number of engineers at the lab. They pick the method, the text it counts, and how long the list should be.

That is why “the” is one card and “differences” is three: one is common in the text they counted, and the other is not.

Sennrich, Haddow, and Birch (2016) brought byte-pair encoding to language models.

Each card number looks up a vector, and the vector carries the meaning



The numbers in the vector are illustrative.

- ▶ The model keeps a table with one row for every card, and each row is thousands of numbers.
- ▶ Those rows are parameters, so they are estimated in training.
- ▶ The same card always starts from the same row, in every sentence.

In our language: the card number is a categorical variable, and its row is a fixed effect with thousands of dimensions.

The list of cards is chosen. What each card means is learned.

Attention lets every token look at the tokens before it

She deposited the check at the **bank**. We sat on the river **bank**.

Both sentences hand the model the same token, number 6922. It needs a different meaning in each.

- ▶ For each token, the model scores how relevant every earlier token is.
- ▶ Those scores become **attention shares** that sum to one.
- ▶ The token's new vector is a weighted average of the others, using those shares.

In our language: a kernel-weighted average, where the kernel is learned from data.

“Deposited” and “check” pull *bank* toward finance. “River” pulls it toward water.

Each token divides its attention among the tokens before it

Updating “bank” in sentence one

Token	Share
She	0.04
deposited	0.38
the	0.03
check	0.33
at	0.05
the	0.03
bank	0.14
Total	1.00

Updating “bank” in sentence two

Token	Share
We	0.05
sat	0.12
on	0.08
the	0.04
river	0.55
bank	0.16
Total	1.00

There is one such column for every token, in every layer. Each column sums to one on its own.

Illustrative shares, not model output.

Attention shares are computed from the text, not estimated

For token i looking at an earlier token j , with x_i and x_j their current vectors:

$$\text{score}_{ij} = (W_Q x_i)'(W_K x_j) \quad a_{ij} = \frac{\exp(\text{score}_{ij})}{\sum_{k \leq i} \exp(\text{score}_{ik})} \quad x_i^{\text{new}} = \sum_{j \leq i} a_{ij} W_V x_j$$

- ▶ W_Q , W_K , and W_V are parameters. They were estimated once, in training, and never change afterward.
- ▶ The shares a_{ij} are not parameters. The model recomputes them for every new piece of text.
- ▶ They are applied to the other tokens' vectors, to mix their information into token i .

Nobody optimizes the shares directly. Training optimizes the matrices, and the shares fall out of the matrices and your text.

What an attention share can and cannot be

Can it be negative?	No. It is a ratio of exponentials.
Can it be zero?	Only for later tokens, which are masked. Otherwise tiny but positive.
Can it exceed one?	No. The shares are positive and sum to one.
Why do they sum to one?	By construction, so that the update is a weighted average.
Is it a probability?	It has the same shape, but nothing is ever drawn from it.
Is it causal?	No. A large share does not mean that token caused the output.

The formula for a share is the multinomial logit formula. The model uses it here to choose where to look, and again at the very end to choose the next token.

Same functional form, two different jobs. Only the last one is a prediction.

On causality: Jain and Wallace (2019), "Attention is not Explanation."

An LLM does one thing: it predicts the next token

“The minimum wage reduces _____”

Next token	Probability
employment	0.41
poverty	0.22
hours	0.11
inequality	0.09
<i>every other token</i>	0.17

Illustrative numbers, not model output.

- ▶ The model draws one token from this distribution and appends it.
- ▶ Then it predicts again, with the longer text as the new input.

An essay, a proof, a Stata do-file: each is thousands of these draws in a row.

Next-token prediction is a multinomial logit

$$\Pr(\text{next token} = k \mid \text{context}) = \frac{\exp(h(\text{context})'w_k)}{\sum_{j=1}^V \exp(h(\text{context})'w_j)}$$

What is familiar

- ▶ The choice set is the vocabulary: V is about 200,000 for GPT-4o.
- ▶ w_k is the coefficient vector for alternative k .
- ▶ It is fit by maximum likelihood.

What is new

- ▶ You do not choose the regressors.
- ▶ $h(\cdot)$ is the transformer, and it is learned jointly with w .
- ▶ There are billions of parameters, not a dozen.

It is a prediction model. Nothing in the likelihood rewards being true, only being likely.

200,000 is the number of choices, not the size of the model

	What it counts	GPT-3 (2020)	GPT-4o (2024)
Vocabulary	distinct tokens it can choose from	50,257	200,019
Context window	tokens it can hold at once	2,048	128,000
Parameters	weights estimated in training	175 billion	not disclosed
Training text	tokens it read during training	about 300 billion	not disclosed

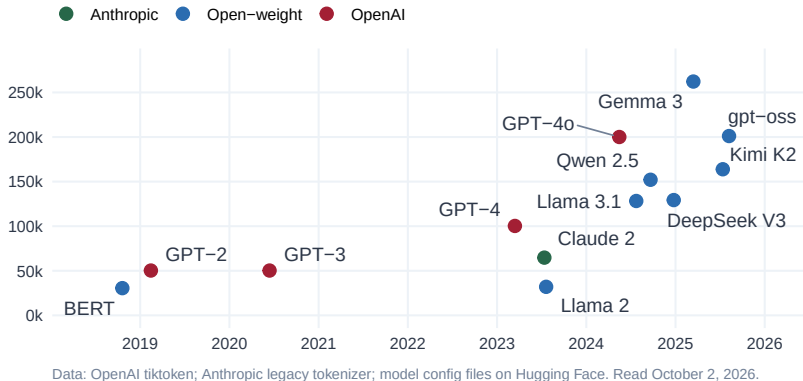
A dictionary has a few hundred thousand entries, and every book is written from them.

The choice set is small. What is enormous is the text it learned from and the weights it takes to choose well.

Sources: Brown et al. (2020) for GPT-3; OpenAI's o200k_base tokenizer and model documentation for GPT-4o.

Vocabularies have grown, but only into the hundreds of thousands

Distinct tokens in each model's vocabulary



Missing: Claude 3 and later, and other new closed models, which publish no tokenizer.

Training a model means estimating its weights

The weights are the parameters of the model, θ : the numbers inside every layer.

- ▶ Training shows the model real text and asks it to predict each next token.
- ▶ The weights are nudged, a little at a time, to make the true next token more likely.

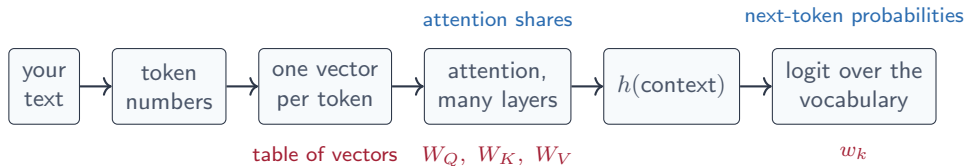
$$\hat{\theta} = \arg \max_{\theta} \sum_t \log \Pr_{\theta} (\text{token}_t \mid \text{tokens before } t)$$

In our language: the objective is the log-likelihood of the true next token, summed over every position in the training text.

There is no closed form, so the maximum is found by stochastic gradient descent.

Once trained, the weights are fixed. When you chat with a model, nothing is being estimated; it is only being evaluated.

One model and one objective; the rest is computed along the way



- ▶ **Red** is estimated: all the parameters are chosen together to maximize one likelihood.
- ▶ **Blue** is computed: the fixed parameters and your text go in, and these come out.
- ▶ Attention shares are a step inside $h(\cdot)$, and the probabilities are the final output.

There is no separate model for attention. Its matrices are estimated by the same next-token likelihood.

Three sets of numbers, and only one of them is “the weights”

	Parameters (“weights”)	Attention shares	Next-token probabilities
Where they come from	maximizing the likelihood	a logit formula inside $h(\cdot)$	the logit formula at the end
What goes in	the training text	W_Q , W_K , and your text	$h(\text{context})$ and w_k
When	once, before you use it	every layer, every token	once per token it writes
Fixed afterward?	yes	no	no
Can be negative?	yes	no	no
Sum to one?	no	yes, for each token	yes, over the vocabulary
Used for	every computation	mixing context into a token	drawing the next token
In the logit	the coefficients	part of $h(\text{context})$	the fitted probabilities

When someone says a model has 175 billion weights, they mean the first column. The other two are computed fresh each time.

Training also runs on hidden human labor

Building the token list needed no human labelers, but training does. **Mary L. Gray** of Microsoft Research, a 2020 MacArthur Fellow, and Siddharth Suri named this *ghost work* in their 2019 book.

- ▶ People label text and images so that a model can learn from the labels.
- ▶ People read the worst material on the internet and flag it, so that a filter can learn to block it.
- ▶ People rate and correct a model's answers after it is trained, which is what makes it polite and useful.

In 2023 **TIME** reported that Kenyan workers labeling abusive text for OpenAI's filter took home less than \$2 an hour.

Sources: Gray and Suri, *Ghost Work* (2019); Billy Perrigo, *TIME*, January 18, 2023.

GPUs do the arithmetic all at once

What the arithmetic is

- ▶ Every layer is matrix multiplication.
- ▶ Billions of weights means billions of multiplications per token.

Why a GPU

- ▶ A CPU has a few fast cores that work one step at a time.
- ▶ A GPU has thousands of simple cores that do the same operation in parallel.

GPUs were built to draw video-game graphics, which is also a large pile of matrix arithmetic.

Training a frontier model ties up tens of thousands of GPUs for months, and every answer you receive runs on them too.

Inside a GPU: billions of switches doing one sum in parallel

- ▶ A chip is a slab of silicon etched with billions of transistors, and each transistor is a switch that is on or off.
- ▶ Wire enough switches together and you get a circuit that multiplies two numbers and adds the result to a third.
- ▶ A CPU spends its switches on a few elaborate units that each do one complicated step after another.
- ▶ A GPU spends its switches on thousands of simple units that all do the same multiply-and-add at the same instant.

A matrix multiplication is millions of multiply-and-adds, and none of them has to wait for any other.

That is the whole match: the transformer is matrix multiplication, and the GPU is a machine for doing one sum a million times at once.

What limits a GPU is memory and heat

One NVIDIA H100

- ▶ It holds 80 billion transistors.
- ▶ It has about 17,000 simple arithmetic cores, against 192 on a top server CPU.
- ▶ It carries 80 GB of memory beside the chip, because fetching the weights is slower than multiplying them.

The heat

- ▶ One H100 draws up to 700 watts.
- ▶ Nearly all of that electricity leaves as heat.
- ▶ A rack of 72 newer GPUs draws roughly 120–130 kilowatts and is cooled with liquid.

Put a few hundred thousand of these in one place and you have a data center measured in gigawatts.

Sources: NVIDIA product pages; DOE National Energy Technology Laboratory; HPE.

Each generation of GPU has cost more than the last

NVIDIA's top data-center GPU	Launched	Price of one chip
K80	2014	\$5,000
V100	2017	about \$10,000
A100	2020	\$10,000–\$15,000
H100	2023	\$25,000–\$40,000
Blackwell B200	2025	\$30,000–\$40,000
Blackwell Ultra B300	late 2025	\$38,000–\$49,000

- ▶ NVIDIA has not published a list price since 2014, so these are reported prices.
- ▶ Each chip also does far more: computation per dollar doubles about every two years.

The chip gets dearer and the arithmetic gets cheaper, both at once.

Sources: Epoch AI hardware data; Jensen Huang, CNBC, March 2024; HSBC via Epoch.

What each generation costs today

GPU	New in	Rent for one hour, October 2026
Blackwell Ultra GB300	this year	from \$10.01
Blackwell B200	last year	\$6.25
H200	two years ago	\$4.38
H100	2023	\$3.40
A100	2020	\$1.82
V100	2017	\$0.97

- ▶ To buy: an H100 is about \$31,000 new and about \$19,000–\$23,000 used.
- ▶ An H100 rented for more than \$8 an hour in 2023 and about \$2 on resale markets by 2025.

How fast these chips lose value is the open accounting question behind the whole build-out.

Sources: getdeploying.com median on-demand prices; Silicon Data; Hashrate Index; CloudZero.

Training runs went from 10,000 GPUs to more than 100,000

Model	Year	GPUs used to train it	Estimated cost of the run
GPT-2	2019	not disclosed	about \$4,000
GPT-3	2020	10,000 V100s, for 15 days	\$2–\$4 million
GPT-4	2023	not disclosed	\$40–\$78 million
Llama 3.1 (Meta)	2024	up to 16,000 H100s	\$53–\$170 million
Grok 4 (xAI)	2025	a 200,000-GPU cluster	about \$390 million
GPT-6 Astra	2026	more than 100,000	\$0.5–\$1 billion

- The low figure counts the hardware used up, and the high figure prices it at cloud rental rates.

The cost of the largest run grows about 2.4 times a year, and most labs do not say what they used.

Sources: Patterson et al. (2021); Meta; xAI; OpenAI briefing, September 2026; Epoch AI; Stanford AI Index; Cottier et al. (2024).

Anthropic reports its compute by company, not by model

What it has said, at the company level

- ▶ It uses more than one million of Amazon's Trainium2 chips to train and serve Claude.
- ▶ It has contracted for up to one million of Google's TPU chips.
- ▶ It also runs on NVIDIA GPUs, including a cluster of more than 220,000.

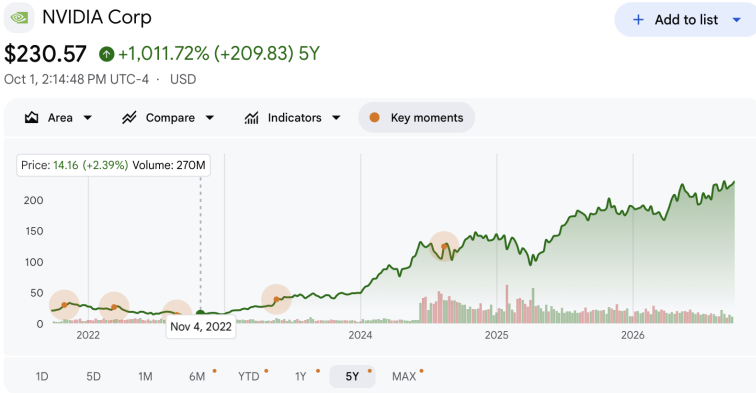
The one cost figure on record

- ▶ Dario Amodei said Claude 3.5 Sonnet cost “a few \$10M's” to train.

Much of Claude is trained on chips that are not NVIDIA's, which is one reason the GPU counts are not comparable across labs.

Sources: Anthropic announcements, October 2025 to May 2026; Amodei (2025), as cited by Epoch AI.

NVIDIA sells the GPUs, and its stock is up tenfold in five years



Source: Google Finance, October 1, 2026. Price \$230.57, up 1,012% over five years.

Compute is the scarce input, and it is rented by the hour

NVIDIA data-center revenue

Fiscal 2022	\$10.6 billion
Fiscal 2023	\$15.0 billion
Fiscal 2024	\$47.5 billion
Fiscal 2025	\$115.2 billion
Fiscal 2026	\$193.7 billion

What a GPU costs

- ▶ One Blackwell chip sells for \$30,000–\$40,000.
- ▶ Renting one H100 costs about \$3.40 an hour.
- ▶ The final training run of GPT-4 cost \$40–\$80 million.

The five largest cloud companies will spend more than \$690 billion on this in fiscal 2026.

Sources: NVIDIA annual results; Jensen Huang, CNBC, March 2024; getdeploying.com median, October 2026; Epoch AI and Stanford AI Index; FactSet, July 2026.

Data centers now show up in the national statistics

- ▶ Data centers used 4.4% of U.S. electricity in 2023, and the projection for 2028 is 6.7% to 12%.
- ▶ The five largest cloud companies will spend more than \$690 billion on capital in fiscal 2026.
- ▶ Debt paid for 32% of that spending by mid-2026, up from 9% two years earlier.
- ▶ Private office construction, the category that holds data centers, was up 30% over the year in August 2026, while manufacturing construction was down 20%.
- ▶ OpenAI's site in Abilene, Texas is planned for 1.2 gigawatts.

This is the largest private investment boom in the country, and it is concentrated in a handful of firms.

Sources: Lawrence Berkeley National Laboratory, 2024; FactSet, July 2026; Census Bureau, October 1, 2026; Epoch AI.

How much of GDP growth is AI? It depends on how you count

Who	Share of U.S. GDP growth	Period
Jason Furman	92%	first half of 2025
St. Louis Fed	39%	first three quarters of 2025
ING	about one third	second quarter of 2026
Goldman Sachs	"basically zero"	2025

- ▶ The high numbers count all spending on computers and software.
- ▶ The low numbers subtract imports, because most of the chips are made abroad.

The spending is enormous either way. How much of it is American output is a measurement question.

Sources: Furman, September 2025; Rubinton and Patro, St. Louis Fed, January 2026; Knightley, ING, August 2026; Hatzius, Goldman Sachs, 2026.

A majority in both parties opposes a data center nearby

Percent opposed	All	Democrats	Independents	Republicans
NBC News, August–September 2026	69	81	71	57
Heatmap, May 2026	71	78	72	63
UMass Amherst, August 2026	65	76	67	52

- ▶ One year earlier the country was split: 43% in favor and 42% opposed.
- ▶ Local opposition blocked or delayed \$64 billion of projects between May 2024 and March 2025.
- ▶ Of the officials publicly opposing projects, 55% were Republicans and 45% were Democrats.

Almost nothing else in American politics polls like this.

Sources: NBC News; Heatmap and Embold; UMass Amherst Poll; Data Center Watch.

The two parties object with different emphasis

Percent who say data centers are. . .	Democrats	Republicans
mostly bad for the environment	62	47
mostly bad for home energy costs	58	44
a worry for the local water supply	67	49
a benefit for technological advancement	54	60

- ▶ Democratic officials stress the environment and the use of water and power.
- ▶ Republican officials stress tax incentives and strain on the grid.
- ▶ In November 2025 two Democrats won seats on Georgia's utility commission, the party's first since 2000, in a race about power bills.

The coalition against them is broad, and the industry that needs them is the one driving investment.

Sources: Pew Research Center, August 2026; AP-NORC and EPIC, July 2026; Data Center Watch.

Closed models lead, and open-weight models run on your own machine

Closed models

Claude, GPT, Gemini

- ▶ The weights are a trade secret.
- ▶ You rent them by subscription or by the token.
- ▶ Your text goes to their servers.
- ▶ They are the frontier.

Open-weight models

Llama, Qwen, DeepSeek, Gemma, Mistral, gpt-oss

- ▶ You download the weights.
- ▶ You run them with Ollama or LM Studio.
- ▶ Your data never leaves your machine.
- ▶ A 16–32 GB laptop runs a 20–30 billion parameter model.

The best open-weight model scored 44 and the best closed model 53 on the same public index in September 2026. Private and free costs you about nine points.

Source: Artificial Analysis Intelligence Index, September 7, 2026.

Effort is a dial for how hard the model thinks

Before the answer you see, a model can write itself a private scratchpad of reasoning tokens.

	Claude	OpenAI
Name of the dial	effort	reasoning_effort
Settings	low, medium, high, xhigh, max	minimal, low, medium, high, xhigh
In the coding agent	/effort high in Claude Code	set per model in Codex

- ▶ Higher effort means more reasoning tokens, more tool calls, and longer answers.
- ▶ You pay for every one of those tokens, in dollars and in waiting time.
- ▶ Low effort is right for a rename, and high effort is right for a proof or a tricky merge.

Same model, same weights. Effort only changes how many tokens it spends before it commits.

You buy intelligence by the token or by the month

By the token (the API)

Dollars per million tokens	In	Out
Claude Fable 5.1	10	50
Claude Opus 5.5	4	20
Claude Haiku 4.5	1	5
GPT-6 Astra	10	50
GPT-6.1 Sol	2	10
GPT-6 Luna	0.10	0.50

By the month (a subscription)

- ▶ Claude Pro is \$20.
- ▶ Claude Max is \$100 or \$200.
- ▶ Claude Code is included.

A million tokens is about 750,000 words.

Use a subscription for your daily work. Use the API when a script must label 300,000 documents overnight.

Sources: Anthropic and OpenAI pricing pages, October 1, 2026.

Introducing AI Agents

From tools you prompt to tools that act

Use agents when the work is valuable, slow, and checkable

My view: delegate to agents when a task satisfies all four conditions.

1. **The output is highly valuable.**

The stakes justify the setup cost.

2. **The task is time-intensive for humans.**

Hours or days of skilled effort, not minutes.

3. **The probability humans do it well is low.**

Difficult to execute at high quality consistently.

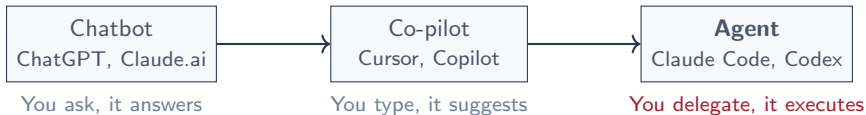
4. **The output can be verified.**

You can inspect, compare, rerun, or source-ground it.

Coding is the canonical example, but this also describes literature review, replication, deck creation, and referee-style review.

Agents are different because they act

A spectrum of AI use, from passive to autonomous:



An agent reads files, writes code, runs it, reads the output, and decides what to do next across many steps.

What the agent actually does

An agent is not just a chatbot with better prose. It acts on your project.

<code>ls data/</code>	list files
<code>rg "minimum wage" .</code>	search text and code
<code>grep -r "treated" code/</code>	search inside files
<code>curl -O https://site.gov/data.csv</code>	download from the web
<code>Rscript estimate.R</code>	run an R script
<code>python audit.py</code>	run a Python script
<code>stata -b do analysis.do</code>	run a Stata do-file
<code>pdflatex paper.tex</code>	compile a paper or deck
<code>git diff</code>	inspect changes
<code>rm -rf data/</code>	delete files, permanently
<code>diskutil eraseDisk ...</code>	format a hard drive

It reads files, writes files, runs commands, sees errors, revises, and tries again.

Reasoning models and agents are merging

Reasoning LLMs

o1, o3, DeepSeek R1, Claude extended thinking

- ▶ Trained to deliberate before answering
- ▶ Pure inference: no action in the world
- ▶ Strong at math, code, logic puzzles

AI agents

ReAct, tool use, Claude Code, Cursor, Codex

- ▶ LLM + tools: read, write, run, search
- ▶ Loop: act, observe, decide, repeat
- ▶ Strong at long, multi-step tasks

2025–26: the merger. Frontier systems increasingly *think* and *act*: plan a task, execute steps, check output, and revise.

A reasoning agent leaves a trace, and the trace is data

What is in a trace

- ▶ the reasoning the model wrote to itself before it acted
- ▶ every command it ran and every file it opened
- ▶ everything it saw come back, including the errors

Why it matters to a researcher

- ▶ Claude Code saves each session to a transcript file on your machine.
- ▶ A transcript is text, so an agent can read hundreds of them and code what happened.
- ▶ This is how I learned what my agents did that their finished output never showed.

The output tells you what the agent concluded. The trace tells you what it did to get there.

A trace from the minimum-wage runs (Kimi, open-weight)

One agent's own account of the choices it made:

"Post-2009 period chosen because the federal minimum wage was constant at \$7.25, creating clean cross-state variation. Callaway–Sant'Anna drops 15 states already treated in 2010 (no pre-treatment data in the sample). Effective sample: 36 states, 16 treated and 20 never-treated."

Every choice is in the trace: the window, the estimator, the dropped states. We extract these from every run.

Closed models summarize the reasoning; open ones show all of it

Model	What you get of its reasoning
Claude (Anthropic)	a summary of the thinking, not the raw text
GPT reasoning models (OpenAI)	hidden, with a summary on request
Kimi, DeepSeek, Qwen (open-weight)	the full raw reasoning

- ▶ Closed labs summarize the raw reasoning, citing competitive advantage and keeping rivals from training on it.
- ▶ What the summary leaves out is unknown.
- ▶ You can raise the effort setting and get more, but you pay for that too.

Sources: Anthropic and OpenAI developer documentation.

So I study both, and I want to close the gap

- ▶ To study how agents behave I have needed all of the reasoning, so I run experiments on open and closed models alike.
- ▶ My running question: can a harness that follows a checklist I have been building make a cheaper model work like an expensive one?
- ▶ Kimi is cheaper, so I want to know whether harnessing it can close the gap.

To study that, I have needed the traces: the trace is how you see what the agent is actually doing.

Claude Code began as a terminal agent

Boris Cherny joined Anthropic in 2024. Late that year he gave Claude direct access to his terminal and asked it to fix his Spotify playlists.

Claude wrote API calls, read its own output, handled errors, and finished the job autonomously. *That was the proof-of-concept.*

- ▶ **Sept. 2024.** Cherny builds the prototype as a side project.
- ▶ **Nov. 2024.** Released inside Anthropic.
- ▶ **Feb. 24, 2025.** Public research preview with Claude 3.7 Sonnet.
- ▶ **May 2025.** General availability.
- ▶ **Nov. 25, 2025.** Claude Code lands inside the desktop app.

Desktop integration of Claude Code is to AI agents what ChatGPT was to GPT — product, not capability, is probably the unspoken hero of this revolution.

Install Claude Code by asking Claude to install it

You have to get Claude Code onto your machine somehow. The install command is the first place phishing finds you.

The trojan-horse problem

- ▶ cloned sites imitating `claude.ai`, GitHub, package docs
- ▶ “copy” buttons that hide a different command than the one shown
- ▶ pasting an unfamiliar install command is voluntary malware

The defensive habit

- ▶ open the desktop app (or another agent you trust)
- ▶ give it the URL you were about to paste from
- ▶ have it inspect domain, repo, command, and explain
- ▶ run nothing until the explanation matches what you expect

Anthropic is currently doing better than most at not getting cloned. That will not last. Build the habit before the arms race catches up.

Claude is not one interface

The terminal is where Claude Code began. It is still the cleanest place to work with project folders, code, data, and reproducible files.

CLI / terminal

- ▶ Project directories
- ▶ File edits and code runs
- ▶ Repeatable workflows

Desktop / Cowork

- ▶ Conversational work
- ▶ Shared context
- ▶ Faster collaboration loops

Read Chris Cornwell's Claude academic guide:

<https://cornwl.github.io/files/claude-academic-guide.html>

Explain which parts apply to my project.

Help me choose CLI, Desktop/Cowork, or both.

Agents are a general-purpose technology

The same agent runs my Callaway–Sant’Anna replication *and* grabs a file off the internet when I need it.

Trivial uses

- ▶ push to my repos
- ▶ edit my website
- ▶ update my CV
- ▶ fetch from the internet

Sophisticated uses

- ▶ replication audits
- ▶ bibliography checks
- ▶ package comparisons
- ▶ many-agent experiments

Most of the daily learning about how to work with agents happens at the trivial end.

Desktop or terminal: pick what fits your brain

Both run the same agent. The choice is about workflow, not capability.

The desktop app (what I reach for)

- ▶ The chatbot and the coding agents sit together, which suits a conversational, thinking-partner way of working.
- ▶ Folders, with sessions inside them, match how I organize my research.
- ▶ The tradeoff: it is slower.

The terminal (what I use for speed)

- ▶ It is noticeably faster. I cannot fully explain why, but it is.
- ▶ The cost is window sprawl; tools like Ghostty help, and some prefer an IDE such as VS Code.

I use both: the desktop app to think, the terminal when speed matters.

Three operating modes

Mode	What it does	Where
Plan	Read-only. Searches, reads, proposes. No state changes.	CLI & Desktop
Action	Edits, runs, writes. Each risky tool call requests approval.	CLI & Desktop
<code>--dangerously-skip-permissions</code>	Edits, runs, writes — no approvals. The agent runs free.	CLI only

Why the third mode matters: the agent goes fast. Launch a long task, walk away, come back to a finished result. Not available in the desktop app. You have given up the per-tool checkpoint, so the safety question is no longer the flag — it is the blast radius of the folder.

Plan mode: iterate on the design, not the files

Read-only: the agent reads and proposes, but changes nothing until you approve.

How Paul Goldsmith-Pinkham uses it:

- ▶ Research the data, then let plan mode design the whole pipeline.
- ▶ It writes the plan to a file, so the design survives a fresh context.
- ▶ Only then execute, building from the plan.

Iterate on the design, which is cheap, before the code, which is not. *Confession: I jump in and try not to drown, so plan mode has never been my habit. If you're more deliberate than me, start here.*

IDEs: open a whole folder and see everything

An IDE (integrated development environment) is a workspace built around your whole project, not one file at a time.

- ▶ You open a folder and see the entire codebase at once, with side panels for files, packages, and a terminal.
- ▶ When the agent edits files you watch it happen: changed files are flagged, and edited lines are marked right in the margin.
- ▶ Examples: VS Code (the common one), Kiro (agent-first), and Positron (Posit's data-science IDE, the RStudio successor).

The CLI is a message in a bottle: fast, but you cannot see the files. An IDE lets you watch the diffs as they land.

The API, and Models You Run Yourself

Keys, credits, and installing an open-weight model

An API key is a billed password for your code

API stands for Application Programming Interface: a way for your code to call the model directly, with no app in between.

- ▶ The key is a secret string that authenticates you and charges the usage to your account.
- ▶ You reach for it when you script the model: many agents at once, an overnight job, or a model your subscription does not include.

A subscription is for you at the keyboard. The API is for your code, running on its own.

Get a key, load credits, paste it in

- ▶ The same three steps at each lab: create an account, add a card, generate a key.
- ▶ **OpenAI:** `platform.openai.com` **Anthropic:** `console.anthropic.com` **Kimi:** `platform.kimi.ai`
- ▶ Credits are pay-as-you-go: load a balance (say \$20) and each call draws it down.
- ▶ The key lives in an environment variable, never in your code and never on a slide.

Treat the key like a card number: it spends real money, so guard it and set a low cap.

Open weights: have Claude Code install it for you

An open-weight model (Kimi, and others) runs through the same tools, and Claude Code can wire one up in minutes. Why bother, when closed models are a few points stronger?

- ▶ You see the raw reasoning: closed labs now summarize the chain of thought, partly to keep rivals from training on it, while open weights hand you the unredacted trace.
- ▶ You can pin the exact model forever, so results replicate; closed models drift and get deprecated.
- ▶ Lower cost, more privacy, and a second model family for external validity.

For studying how agents behave, an unobfuscated trace is worth a few points of capability.

Live demo: buy a key, run an open model

1. Open `platform.kimi.ai`, add a few dollars, and create a key.
2. Hand the key to Claude Code as an environment variable.
3. Ask it to run one Kimi agent on our running example.
4. Open the transcript and read the raw reasoning it wrote.

The point is not the estimate. It is that you can stand up your own model and watch it think.

Code Review, and Claude in the Loop

Open a pull request, review it, and let the agent answer

First, the plan: review on GitHub, fix in the terminal

Before the mechanics, here is what we are going to do — and which screen to watch.

The browser — GitHub

- ▶ my repo:
scunning1975/mixtape_harness
- ▶ where I review the change, line by line, and leave comments

The terminal — Claude Code

- ▶ where the *agent* proposes the change, then answers my comments and pushes fixes

The round trip: I make a change → Claude opens it as a pull request → I review it in the browser → I hand the comments back → it fixes and pushes → I refresh.

I review in the browser; the agent works in the terminal — same pull request, both sides.

Code review: checking the diff, line by line

At a large tech firm

- ▶ You do not push straight to the codebase; you open a code review, a “cr,” in an internal tool (`code.techfirm.dev`).
- ▶ A reviewer expands the diff and comments on the exact lines.

What I have learned the large firms do.

On GitHub

- ▶ The same thing is a **pull request**.
- ▶ The “Files changed” tab is the diff, and you comment inline on any line.

The new move: put Claude on both sides. It opens the pull request, and it answers the comments.

The pull request is the review

- ▶ Claude opens it: `gh pr create`, writing the title and body from your commits.
- ▶ GitHub's "Files changed" tab is your review: the full diff, expandable, with inline comments.
- ▶ You read it in the browser like any reviewer and leave comments where something is off.

Review is where verification becomes a conversation, line by line.

Hand the comments back to Claude

In the cloud

- ▶ Install the GitHub app once:
`/install-github-app`.
- ▶ Then `@claude` in a pull-request comment, and it pushes the fixes as new commits.

On your machine

- ▶ `claude` reads the thread with `gh pr view 42 --comments`.
- ▶ It edits the files, and you watch the diff update.

You are the reviewer. Claude is the author who never tires of revisions.

Let an agent review the diff first

- ▶ `/code-review` checks your current diff for real bugs; `/code-review 42` reviews pull request 42.
- ▶ `--fix` applies the findings to your files; `--comment` posts them as inline comments on the PR.
- ▶ `/code-review ultra` escalates to a multi-agent review in the cloud.

A second agent reviews the first. You still read it, but the easy catches are already flagged.

/loop: the agent on a timer

```
/loop 10m "pull the repo, read new review comments, address them, and  
commit"
```

- ▶ It runs that prompt on a schedule, between your turns, while the session stays open.
- ▶ Honest limit: the loop lives only while the terminal is open. It is not unattended automation.
- ▶ For round-the-clock, use a Routine or a GitHub Action on a schedule instead.

You can watch the agent cycle through a review in real time. Just know it stops when you close the terminal.

Live demo: open a PR, review it, loop

1. Branch, make a small change, and let Claude run `gh pr create`.
2. Open the pull request in the browser and leave one inline comment.
3. Back in Claude Code: `gh pr view 42 --comments`, then fix and push.
4. Start `/loop 10m` and watch it pick up the next round on its own.

We run this on my own repo, live.

Directing the agent

Four standing tools turn a general agent into yours. They load in every session, so you set them once.

- ▶ **CLAUDE.md and AGENTS.md:** what it knows. The briefing it reads every time.
- ▶ **Skills:** what it can do. Reusable procedures you call by name.
- ▶ **Hooks:** what it must do. Automatic checks that fire around its actions.

Context, capability, and guardrails. The rest of this part is how you direct an agent without writing code.

CLAUDE.md is a standing briefing, not a law

Claude Code loads this file into its context at the start of every session in the folder.

Put in it what stays true

- ▶ what the project is and where things live
- ▶ how to compile, test, and check the work
- ▶ your rules for code, writing, and style
- ▶ what to ask you before doing

Know its two limits

- ▶ It is read every session, so every line costs attention. Keep it short, and keep changing status in a separate file.
- ▶ It is a request, not a guarantee. A rule that must never be broken needs a hook or a permission setting.

In a project folder:

```
claude
```

```
Read this repo and draft a CLAUDE.md for how we should work.
```

```
Ask me before writing the final version.
```

```
# (or type /init for a first draft)
```

Start with CLAUDE.md. Add a skill when a task repeats, and a hook when a rule must hold.

Everyone else calls this file AGENTS.md, and Claude Code now reads it

AGENTS.md is the same idea under a shared name. Codex, Cursor, Copilot, and Gemini all read it.

Your project has	Claude Code reads
only AGENTS.md	AGENTS.md
both files	CLAUDE.md only, unless you change one setting
both files, with the setting on	both, CLAUDE.md first

- ▶ This arrived in Claude Code in September 2026.
- ▶ To read both, open /config and set *Project instructions* to read both files.
- ▶ If coauthors use other agents, write one AGENTS.md and let every tool share it.

The file name is a convention. What the file can and cannot do is the same either way.

Prose does not enforce

CLAUDE.md, AGENTS.md, prompts, and skills are all prose, and prose is cheap talk.

- ▶ An LLM is like a respectful child who regularly sneaks out of the house.
- ▶ When caught, it apologizes and swears it will never do it again.
- ▶ Then it usually does it again within a few minutes.

Anthropic's own documentation says Claude treats these files as “context, not enforced configuration.”

Be wary of markdown as a control. The only way to constrain an action is to make it impossible, and hooks can do that.

Skills are reusable research routines

CLAUDE.md tells the agent how this project works. A **skill** tells the agent how to do a recurring task. The ones I made because I do them constantly:

Production

- ▶ /beautiful_deck
- ▶ /tikz
- ▶ /split-pdf

Verification

- ▶ /referee2
- ▶ /blindspot

Literature audit

- ▶ /bibcheck

This is how one-off prompting becomes a workflow.

PDFs are still a bottleneck

It feels like a PDF contains text. Often it does, but not in the way a markdown file contains text.

PDFs are closer to layout instructions: glyphs, coordinates, columns, figures, footnotes, headers, OCR artifacts, and broken reading order.

For small papers, agents can often manage this. For giant PDFs, scans, appendices, or books:

- ▶ the chat may slow down or die
- ▶ the context may fill with layout debris
- ▶ the agent may lose the thread
- ▶ your work can disappear with the session

Do not assume “it can read PDFs” means “it can reliably reason over any PDF.”

MixtapeTools: /split-pdf

/split-pdf is my answer to giant PDFs.

Instead of asking one agent to swallow the whole file, the workflow decomposes the document:

1. split the PDF into smaller chunks
2. send chunks to parallel agents
3. write markdown summaries for each chunk
4. run a final synthesis agent over the summaries
5. save a definitive markdown representation

The goal is not just reading. The goal is a durable intermediate object the next agent can inspect.

Decks are the gateway drug

Slides are valuable. When I graduated in 2007, all of us shared Imbens and Wooldridge's NBER lectures. They came out before *Mostly Harmless*. No textbook — just slides. They were the bible for an entire cohort.

Hard to make well. Extremely time-consuming, sometimes infinitely so. Especially when they require TikZ — a powerful, dense graphics package Claude knows backwards and forwards.

I usually make bad slides because I cannot stop tinkering long enough to do what I actually like doing: thinking about how the talk should land.

Verifiable in front of you. Good or bad, the slide is right there. You can see it.

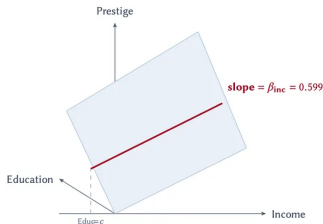
Culturally different from writing. Publishers give you slides. We share each other's slides. We do not even know how to cite each other's slides.

So slides are the place where delegating to an agent feels lowest-stakes and highest-leverage simultaneously. Start there.

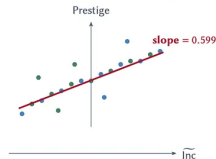
The FWL diagram I could not see in my head

The BLP plane slice and FWL both recover slope $\beta_{\text{inc}} = 0.599$ — from the same partial variation

Long BLP plane: fix Education, vary Income



FWL: Prestige vs $\widetilde{\text{Inc}}$



Both red lines have slope $\beta_{\text{inc}} = 0.599$ — the BLP plane slice and FWL are the same object

Last week: fixing Education at any value c gives a line through the BLP plane with slope β_{inc} . FWL extracts that slope from the 2D scatter by removing Education's variation.

`/beautiful_deck`

`/beautiful_deck` is built around a simple idea: a deck is not a document dump. It is rhetoric under time pressure.

`https://github.com/scunning1975/mixtapetools`

Give Claude the repo link and say:

Read the MixtapeTools repo.

Install or adapt the `beautiful_deck` skill for this project.

Use the `rhetoric-of-decks` essay as the standard.

This is production, but the standard is rhetorical: pathos, ethos, logos, low density, high impact.

Decks are rhetoric, not document dumps

/beautiful_deck sits on top of a short essay — *The Rhetoric of Decks*.

The standard is not coverage. It is persuasion under time pressure:

- ▶ titles are assertions, not labels
- ▶ one idea per slide
- ▶ ethos, pathos, logos — in that order, every time
- ▶ beauty is function: every element earns its place
- ▶ compile clean. zero warnings. always.

The skill is not building slides. It is teaching the agent what a good talk looks like.

`/beautiful_deck` **compiles clean.** `/tikz` **sees what it cannot.**

`/beautiful_deck` loops on overfull boxes, font warnings, undefined references — the things `pdflatex` catches. It will not stop until they are gone.

But `pdflatex` **never warns** that a label sits on a Bezier curve, a legend overlaps the plot, or an axis title is clipped at the figure edge. `ggsave` does not refuse to write the PNG. The producing agent declares the figure done.

LLMs have poor spatial reasoning over rendered output. Eyeballing is unreliable. So `/tikz` replaces it with two mechanisms:

- ▶ **Math mode:** Bezier-curve geometry on the source — chord length, bend angle, label safe zone
- ▶ **Visual mode:** multimodal vision on the rendered PNG, looking for the same three errors

Compile-clean is necessary. Visually clean is what makes the figure beautiful.

Example: clean a chaotic project folder

Many research projects are not clean pipelines. They are sediment.

- ▶ old scripts with unclear order
- ▶ duplicated data and output files
- ▶ tables whose source is no longer obvious
- ▶ folders organized by memory, not documentation
- ▶ projects that become emotionally costly to reopen

A safe first agent task: inventory, map, and propose. Do not move or rewrite yet.

Example: my Downloads folder

This is not a research project. It is where research materials, teaching files, installers, images, and forgotten data extracts accumulated.

Total files	32,402
Top-level items	336
Total size	104 GB
Date range	May 2010 → today
Top-level folders	94

Top-level mix: 48 PNGs, 39 PDFs, 37 HEICs, 33 .md, 13 MP4s, 13 CSVs, 12 .txt, 11 MP3s, 7 forgotten .dmg installers, 6 .docx. The .do/.tex/.dta files didn't leave—they sank into subfolders (1,193 .tex, 1,428 CSVs, 1,041 .dta tree-wide).

336 top-level items is already too many for Finder. So the first job is not cleanup. It is diagnosis.

The mess has a shape

The single largest item is a 24 GB folder named 154561-V1 — not the IPUMS extract I assumed, but a forgotten openICPSR replication package stuffed with patent data.

- ▶ 19 GB in the 2026-03 folder — mostly a 9.5 GB `cps_00025.dat`, *that* being the IPUMS CPS extract
- ▶ 9.3 GB of teaching folders
- ▶ eight “Episode” MP4s at 2.2–3.8 GB each
- ▶ the five biggest items account for about 64 GB of 104 GB

This is exactly the kind of task where an agent can make the invisible structure visible before anyone touches a file.

Two folder workflows

Clean an old project

Read this folder. Do not modify files.
Map data, code, outputs, and pipeline
order.

Flag inconsistencies.

Start correctly

`github.com/scunning1975/mixtapetools`
`/newproject my-project`

Creates a standard scaffold: CLAUDE.md,
README.md, code/, data/, output/,
documents/, decks/, notes/, and
progress_logs/.

The shared principle: make structure visible before asking for judgment.

Hooks: the rule the agent cannot skip

When prose is not enough, you reach for code

When prose fails, you reach for a hook

Recall: CLAUDE.md, skills, prompts — all prose, all *cheap talk*. The model can skip any of them under load.

- ▶ CLAUDE.md — what the agent *knows*. A briefing.
- ▶ **Skills** — what the agent *can do*. A procedure you call by name.
- ▶ **Hooks** — what the agent *must do*. Code the harness runs, whatever the model decides.

How to know: reach for a hook only when a rule must hold *every time* — when “please don’t” is not good enough.

What a hook is — and who made it

A **hook** is a shell command the harness runs *automatically* at a fixed moment in the agent's lifecycle — not when the model chooses, but every time.

- ▶ **Who.** Anthropic's Claude Code engineers shipped them in 2025 — a feature of the tool, not something the model writes for itself.
- ▶ **The moments.** PreToolUse (before a tool runs), PostToolUse (after), plus session start/stop, prompt-submit, pre-compact.
- ▶ **Why.** The model is probabilistic; a hook is deterministic. It fires whether the agent cooperates or not.

A skill is advice the agent may take. A hook is a gate it cannot walk around.

How a hook works — and the simplest one

You register it in `.claude/settings.json`: a *matcher* (which tool) and a *command*. The event arrives as JSON on stdin; **exit 0 allows, exit 2 blocks** and hands your message back to the agent.

```
"PostToolUse": [{ "matcher": "Edit|Write",  
  "hooks": [{ "type": "command",  
    "command": "black $CLAUDE_PROJECT_DIR" }]}]
```

That is a whole hook: after every edit, run the formatter. No script needed. **Any** language — a bash one-liner, Python, Node. Mine live in `hooks/*.py`, registered in `settings.json`.

What makes a great hook

- ▶ **Structural, not textual.** Match a named field — the file *path* — not words in the agent's text. A constructed path defeats any string-matcher by construction.
- ▶ **Narrow.** Near-zero false positives. A hook that cries wolf trains you to ignore it.
- ▶ **Fail-open on errors.** A malformed event must never wedge the session — but fail *closed* on the thing it guards.
- ▶ **Speak plainly.** Hand back one clear sentence, not a cryptic `errno`.
- ▶ **A tripwire, not the whole wall.** The hook is the fast top layer; the real guarantee lives deeper.

My five hooks at a glance

Hook	Fires on	Enforces
protect-raw-data	Pre: Edit/Write	raw data is immutable — never overwrite data/raw/
no-fabricated-exhibit	Pre: Edit/Bash	no figure from made-up data (unless a labeled Monte Carlo)
no-offbook-exhibit	Pre: Bash	no figure drawn inline in a shell — only named scripts
deck-from-pipeline	Post: Edit/Write	no decked exhibit unless a wired script rebuilds it
no-stale-canon	Post: Edit/Write	warn if a “canon” exhibit is missing, unwired, or stale

Every one guards the same chain: real data → named code → exhibit → deck.

protect-raw-data: **the data is immutable**

Fires before any Edit/Write whose target is an *existing* file in data/raw/. Blocks it — but adding a *new* raw file is fine.

- ▶ **The rule:** raw source data is never modified. Every transform *reads* raw and *writes* to data/derived/.
- ▶ **Structural:** it resolves the real path (symlinks, .., case) — not a word-scan.
- ▶ It hands back a sentence, not an errno: “read it; write transformed output to derived/.”

This is the fast top layer. The real wall is a root-owned vault — which is where we'll push on it, live, at the end.

no-fabricated-exhibit: **never a made-up number**

Fires when a figure/table script contains synthetic-data language *and* a random-number generator — on Write, and on the Bash that runs it.

- ▶ **The incident (2026-07-17):** a teaching slide drew *illustrative* synthetic data with a footnote. It slipped past every other guard because it came through a figure script.
- ▶ **Carve-out:** a labeled Monte Carlo (a # MONTE CARLO header) passes.
- ▶ A footnote is not a defense. The number must trace to real data.

My first rule, turned into a gate: never present a figure that doesn't trace to real, unmodified data.

The pipeline pair: every exhibit must be reproducible

no-offbook-exhibit (before a Bash run). Blocks drawing a figure *inline* in a shell — `python -c "...savefig..."`, a heredoc. A picture with no producer file can't be re-run or audited. Every exhibit is born in a named script. `deck-from-pipeline`

(after an edit to a deck). Blocks a deck that references a figure *no script wired into* `run_pipeline.sh` produces. "Ran once in chat" is not enough. **The incident: a whole outcome built ad-hoc in chat, decked anyway — caught only by memory. Now the machine catches it.**

no-stale-canon: the certainty fence

Fires when I mark a stage's exhibits "canon" — done. It checks each one and *warns* (does not block) if it is:

- ▶ **missing** — the file isn't on disk
- ▶ **unwired** — no pipeline script rebuilds it
- ▶ **stale** — the figure is *older* than the script that makes it

Why it exists, in my words: *"if I'm going to make 5× as much stuff, I need to be 5× as sure it's right."* Hanging a figure that was never remade is exactly the silent failure my memory can't catch. The hook can.

“The problem with hooks is you have to debug them”

An engineer at Netflix told me that — half-joking. His point: skills feel easy, so we love them. Hooks are *code*.

And he's right. A hook runs silently, outside the conversation. When it misfires it blocks the wrong thing, or fails with a cryptic error — and you debug it like any program.

- ▶ `protect-raw-data` took a three-agent red team — and two whole arms I built and then *deleted*.
- ▶ every hook here is a few rounds of false positives I had to sand off.

Skills are prose you write once. Hooks are software you maintain.

But the debugging is the price of a guarantee

Here is what the joke hides: a hook can enforce *precisely because* it is code. Skills feel easy because they're cheap talk — and cheap talk is what fails under load.

- ▶ a hook fails **loud** — a block, right now. A skill fails **quiet** — a plausible wrong number you find months later.
- ▶ debugging the hook is how I learned where the *real* wall is: the raw-data hook taught me the guarantee lives in the kernel, not the script.

Pay the debugging tax for the few rules that must hold. Leave everything else to prose.

Live: let's try to break the raw-data hook

Now we debug one together — by attacking it. I point aggressive agents at `protect-raw-data` and tell them to overwrite a raw file any way they can.

- ▶ constructed paths, symlinked parents, case tricks, a heredoc that writes then runs
- ▶ we watch which get blocked — and which expose a gap

When I did this before, a three-agent red team *broke the clever version*. The fix wasn't a cleverer hook — it was a *smaller* one, plus a root-owned vault. **A textual guard loses to a constructed path by construction.** The kernel is the guard that can't be hacked.

3

Verification

Verification is the new bottleneck

Agents make production cheap enough to separate from verification.



The bottleneck moves. The hard part is no longer producing first drafts, first regressions, or first summaries.

The hard part is designing verification systems that survive cheap production.

Verification systems have design ingredients

Do not ask whether one output “looks right.” Ask how the workflow makes errors visible.

Useful ingredients:

- ▶ independent starts
- ▶ narrow assignments
- ▶ adversarial roles
- ▶ cross-language checks
- ▶ forced comparison against source material

The unit of trust is the workflow, not the single answer.

Audits are one way to buy verification

Four empirical-research audits I use as verification tools:

1. **Literature audit** — one agent per cited paper, checking accuracy against source text.
2. **Rhetoric audit** — one agent turns a paper into a talk; another judges whether the story is faithful.
3. **Behavior audit** — many agents become experimental units, revealing how agents behave under constraints.
4. **Implementation audit** — independent agents re-run the same analysis across languages or packages.

The goal is not audit for its own sake. The goal is confidence that the claim, code, citation, or estimate is right.

Borrow templates; use what you actually understand

Two public starter templates, both worth a look:

- ▶ **Clo-Author** (Hugo Sant'Anna) — per-paper pipeline: lit discovery, paper review, methods checks, writing, submission. hsantanna.org/clo-author/
- ▶ **Claude Code workflow** (Pedro Sant'Anna) — per-researcher OS: slides, papers, replication, quality gates. psantanna.com/claude-code-my-workflow/

What I actually do day to day:

1. Drop the paper into **Refine.ink**, download the markdown report, walk through revisions with Claude. Pricey, but a desk reject is more expensive.
2. Use my own /referee2 constantly. Not because it's better — because I built it, I understand it, and it conforms to my objectives.

Borrow the template; build the routine. The thing you actually use needs to be the thing you understand.

MixtapeTools: /referee2

/referee2 is my verification routine for work I do not want the original producing agent to grade.

The core idea is independence:

- ▶ a fresh agent reads the artifact cold
- ▶ it checks code, tables, claims, or slides against source material
- ▶ for code, it can reimplement in another language
- ▶ for decks, it can audit rhetoric, density, compilation, and fidelity

The original agent can explain itself. A fresh verifier is more likely to catch what the first one normalized.

OLS is deterministic, so three languages must agree

/referee2 rebuilds the pipeline in Stata, R, and Python — then compares the numbers.

The logic is that OLS is not a random process. It minimizes the sum of squared residuals, $\hat{\beta} = (X'X)^{-1}X'y$ — a fixed function of the data.

- ▶ So coefficients, standard errors, t -stats, and the F are the *same real numbers* regardless of language.
- ▶ Agreement to six decimals across all three $\Rightarrow$ the pipeline is right.
- ▶ Disagreement is never noise. It is a bug, or a named package default (df adjustment, robust-SE flavor, missing-value handling) — and the audit says which.

LLM coding errors are orthogonal across languages. Three independent implementations converge only when none of them erred.

Save the work before the context dies

Long agent sessions fail. The antidote is to make the project remember what happened.

Write a progress log in `progress_logs/`.

Summarize everything we have done since the last update:

- decisions made
- files read
- files changed
- remaining questions
- next safe steps

Then a fresh agent can re-enter the folder, read the markdown logs, and continue without relying on the vanished chat.

Progress logs turn fragile conversations into durable project memory.

I don't picture my work — I remember it with my hands

I have *aphantasia*: no mental images. When I recall the past — even my own memories — the screen is black. I get the story and the feeling, never the picture.

So for twenty years I never *saw* my projects. I knew them procedurally:

- ▶ repetition — opening the same files every morning
- ▶ the focus zone I'd drop into, picking up from yesterday
- ▶ a map of the project imprinted by hand, over years

The projects I worked longest, I knew best. Memory was muscle, not image.

At arm's length, the muscle memory never forms

Agents moved me out of production. I'm faster — and I never touch the files.

So the instincts I took for granted are simply gone. The basic questions come back:

- ▶ which file made this figure?
- ▶ are these outputs current, or stale?
- ▶ where am I in the project?

It's like piloting a submarine by sonar after a career steering the ship by eye.

A day's context window has to be wiped — on purpose

After a full day, the window is enormous. The model starts to drift: it forgets, repeats itself, spawns stray files, loses the thread.

So `/clear` isn't optional — it's hygiene. But clearing erases the day.

If I have to wipe the conversation, the memory has to live *outside* it — on disk.

`/sleep`: write it all down before the window dies

The night tool. Its audience isn't me — it's *tomorrow's* me, who will remember nothing.

- ▶ a dated log in `audits/` — decisions, changes, corrections, open threads
- ▶ `STATE.md`, the project's working memory — objective, in progress, what's next
- ▶ a *state-of-mind* note — energy, what's settled, what still carries weight

`/sleep` writes exactly what `/amnesia` will read.

`/amnesia: rebuild the map every morning`

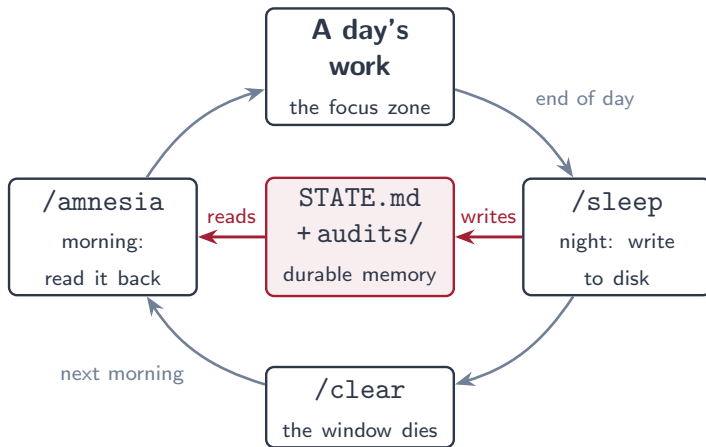
The morning tool. It reads last night's `STATE.md` and newest log, then reorients me:

- ▶ a short summary — objective / just done / in progress / next
- ▶ a flip-through deck at `reorient/` for the visual reload
- ▶ then it walks me back in, question by question, until I'm oriented

Always built from the current files — never stale, regenerated every time.

`/amnesia` answers “where am I?” — and because `/sleep` ran last night, the answer is true.

The loop: the conversation dies, the memory doesn't



/clear wipes the window every day. The disk is what survives it.

4

Mixtape Harness

Memory keeps me oriented. A harness keeps me honest.

The /sleep-/amnesia loop solves one failure: *where am I?*

It does not solve the other. With an agent producing faster than I can check, I *skip steps* — and never feel the skip.

So I built a harness: a checklist for causal inference, turned into machinery, that will not let me run ahead of my own verification.

Experts skip steps — and AI makes the skip silent

Gawande, The Checklist Manifesto. It is not novices who skip steps under load. It is *experts*. Overconfidence scales *with* skill — which is why pilots and surgeons still use written checklists.

What AI changed. Producing and verifying used to be one motion — I understood the code because I wrote it. Now:

- ▶ the agent writes 200 lines in one shot
- ▶ I can approve work I did not produce and do not fully understand
- ▶ the agent is confident even when wrong — so the skip is silent

The harness re-couples production and verification — it restores what working through agents took away.

Rubin: design trumps analysis

Rubin (2008), For Objective Causal Inference, Design Trumps Analysis.

Objectivity is not rescued at the analysis stage with a clever regression. It is *built at the design stage, before you ever see the outcome*.

- ▶ a randomized experiment is trusted because it is outcome-blind — you *cannot* tune it to the answer
- ▶ an observational study earns that trust only by imposing the same blindness: reconstruct the experiment, find the assignment mechanism, balance covariates — outcomes hidden

“Final outcome data cannot be used in design without compromising the objectivity of the study design.”

The design happens before you look at the outcome

Rubin's discipline, in order:

1. **Name** the hypothetical experiment the data pretend to be.
2. **Find** the assignment mechanism — who was treated, and why?
3. **Hide** the outcomes; design on covariates only.
4. **Balance** — and where there is no overlap, *refuse* to infer.
5. **Only then** analyze. The smallest, last step.

The harness keeps the outcome “not available at this stage” — on purpose, the way randomization does it for free.

The same discipline, written as checklists

Imbens. His surveys — including Imbens & Xu's (2024) reassessment of LaLonde — make the case in the selection-on-observables world: check overlap, check balance, run sensitivity, *systematically*. Design first.

The Mixtape. Every chapter ends the same way: a checklist, or practical do-this-then-that tips — one per research design.

The harness is that tradition condensed into something an agent and I walk together, top to bottom.

The harness: Rubin's line, turned into machinery

`github.com/scunning1975/mixtape_harness` — clone it, point it at a study, it enforces the discipline.

- ▶ **The law** — `CLAUDE.md`: zero-error as a *constraint*
- ▶ **The checklists** — the design steps, gated
- ▶ **The stages** — one room per step, re-enterable
- ▶ **The dashboard** — live, reads disk, colors drift
- ▶ **The instruments** — `/pipeline`, `/referee2`, `/covariates`
- ▶ **Working memory** — `STATE.md`

The estimator will not run until the design steps before it are signed off. That gate is the whole point.

The pipeline is the ground truth

Every table and figure in the paper is produced by *pipelined code* — scripts that, at the end, re-run from raw data to rebuild the entire analysis.

- ▶ nothing in the paper is hand-made or remembered — it is *regenerated*
- ▶ every figure traces back to the exact script that made it
- ▶ `/pipeline` re-derives all of it, so “is this output stale?” has an answer

If it is not produced by the pipeline, it is not evidence. Provenance is the antidote to amnesia.

The diff is the unit of observation — but I am not Paul

In the IDE, with code review (a PR, or gh), I watch the *diffs* — every change the agent makes.

Paul Goldsmith-Pinkham: “the diff is the unit of observation.” Review every one.

Honestly, for me that is not quite true. Paul has the memory and the spatial instinct to hold the whole diff-space. I don't — so I do what I *can*: I follow one thread, aggressively. And my first thread is always the same.

Bite. Before anything downstream: did the treatment actually *do something*?

Checklist: target, then bite

1 · Target estimand. Write it in potential outcomes — ATT? group-time? — for a named population, with a weighting rule. Decide it *first*; every later step serves this one parameter.

2 · Bite. Show the treatment's first stage: did it move the thing that is supposed to move the outcome? Maps, time series, the assignment mechanism — *shown before the outcome*.

Bite sits on the design side of Rubin's line. Outcome last.

Checklist: covariates & balance, then sample shares

3 · Covariates & balance. Choose the X that drive trends in the untreated outcome $Y(0)$ (Heckman-Ichimura-Todd). Check overlap; a normalized difference > 0.25 is imbalanced (Imbens & Rubin).

4 · Sample shares. Count treated units by cohort. Callaway-Sant'Anna weights by these shares — so one big cohort can quietly dominate the aggregate.

Every N here must reconcile with the bite map — no orphan units.

Checklist: outcome trends, then power

5 · Outcome trends. Plot mean $\bar{Y}$ by group — *pre-treatment only*. Do the groups move in parallel before treatment? *Do not peek at post-outcomes yet* (Rubin).

6 · Power / MDE. What is the minimum detectable effect, given clusters, periods, and serial correlation (Bertrand-Duflo-Mullainathan)? Say what a null would — and would not — mean.

Power is recorded *before* the estimator runs, so a null is interpretable later.

Checklist: estimator & event study, then falsification

7 · Estimator & event study. Lock the sample (/referee2 drift), then run — Callaway-Sant'Anna preferred. Plot the event study with a universal baseline.

8 · Falsification & sensitivity. Placebo group, placebo outcome, pre-trends — then Rambachan-Roth: how large a trend violation would it take to overturn the result?

The estimator is *gated*: it will not run until steps 1–6 are signed off.

My first thread, always: does the law bite?

Rhode Island, with Manisha Shah. In 2003 a court ruling accidentally decriminalized *indoor* sex work. Before any downstream outcome I asked one question: *did that event actually disrupt the market?*

- ▶ 15–20 qualitative interviews — police chief, head of vice, ED doctors, the FBI, the state-police statistician, newspaper owners, the health department
- ▶ newspaper data, arrest data
- ▶ diff-in-diff and synthetic control — did 2003 “bite”?

Bite first: can I find *any* evidence the law did what it was supposed to — before chasing the downstream effects economists love?

Same move, now with Brazil: Dias & Fontes (2024)

Before homicides, before deaths of despair — *did the reform bite?*

Dias & Fontes (2024), *AEJ: Economic Policy*. Brazil's 2002 psychiatric reform rolled out **CAPS** — community mental-health centers — municipality by municipality through 2016.

- ▶ 5,180 municipalities, staggered adoption — a textbook staggered DiD
- ▶ care shifts from the *institution* to the *community*
- ▶ outcomes: deaths of despair, and — the headline — *homicides*

It is also the running example inside the harness — so the shape of the analysis is already laid out.

The bite is two-sided: care up, institutions down

5-year ATT (per 10k)	Effect	vs. baseline
<i>Community care goes UP</i>		
Outpatient MH procedures	+264.2	+113%
MH professionals	+0.83	+46%
<i>Institutional care goes DOWN</i>		
Psychiatric admissions	-0.95	-7.5%
long-stay schizophrenia	-0.82	

The same policy pushes one margin up and another down. For homicides, the causal first stage is the *admissions drop* — lost incapacitation.

Only one outcome moves — and Penrose explains it

5-year ATT (per 10k)	Effect	
Homicides	+0.149	+7.7% (headline)
Suicide	0.002	ns
Deaths of despair	−0.022	ns
Alcohol-related	−0.018	ns

Penrose's hypothesis. Move the crime-prone severely ill out of incapacitating institutions into under-resourced community care, and violence rises — and the admissions drop is concentrated exactly in long-stay schizophrenia.

Deaths of despair are a true zero; homicides are the one thing that moves.

Every stage ships as a pull request

As we walk the checklist, each stage is a diff you can see — the pipeline script, plus the figure or table it produces.

- ▶ the **unit** is one exhibit — the bite figure, the balance table, the event study
- ▶ the **PR** is the review — you read the script diff *and* look at the rendered output
- ▶ the **gate** is the merge — the next stage does not start until this one is approved

Figures are where errors hide in plain sight. The loop exists to catch the exhibit that renders cleanly but is wrong.

The loop in action: fixing the bite figure

The first exhibit is the bite — psychiatric admissions down, outpatient care up. Watch one round:

1. Claude writes the pipeline script, renders the figure, and runs `gh pr create`.
2. I review the diff and comment on the exact lines: *“drop $g - 1$, label the axis per 10k, trim p -score > 0.995 .”*
3. `/loop 10m` picks up the comments, edits the script, re-renders, and pushes.
4. The diff updates, the figure is right, I merge — and the gate to Step 3 opens.

The figure compiled fine the first time. It was still wrong. That is exactly what the loop is for.

Let's build it together — and watch how I work

We open the dashboard and walk the checklist on the Brazil data, in order:

1. establish the **bite** — admissions down, outpatient up
2. design outcome-blind — target, covariates, balance, trends
3. *then* the estimator, the event study, the falsifications
4. each one ships as a **pull request** — you see the diff, you approve the merge

Bite first, outcome last — Rubin's line, enforced by the harness, with you watching every step.

Could I replicate the PNAS labeling for \$11?

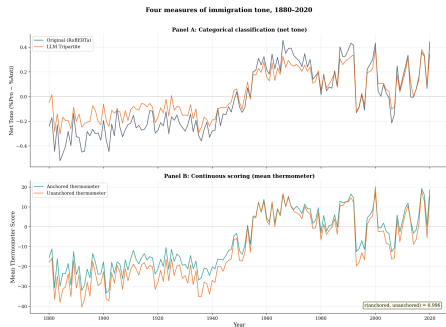
I wanted to see if I could replicate Card, Boustan and Abramitzky (PNAS 2022) on 140 years of US Congressional speeches — but using OpenAI zero-shot instead of their fine-tuned RoBERTa.

What they did. Trained RoBERTa on $\sim 7,500$ speeches hand-labeled by roughly a dozen Princeton students. Probably tens of thousands of dollars and weeks-to-months of human time. *Cost-prohibitive for most resource-constrained researchers.*

What I did. Wrapped each segment in a short prompt, zero-shotted through the OpenAI Batch API on gpt-4o-mini. The hard part is getting the JSON formatting right — which Claude is very good at handling.

Cost. About \$11 and one hour. Then I had Claude produce the comparison against their published labels.

Independent labeling tracks Card–Boustan–Abramitzky across 140 years



Net-tone trajectory: zero-shot gpt-4o-mini on 305,000 Congressional speech segments vs. Card-Boustan-Abramitzky's RoBERTa (PNAS 2022). No fine-tuning. No human annotators.

This is what verification looks like when the production process is replaced.

Independent implementations turn disagreement into evidence

Replicating analysis in a different language is not about new data. It is about verifying the original implementation.

The logic: syntax errors are probabilistic and language-specific.

- ▶ A merge error in R will not reproduce identically in Stata.
- ▶ A package-specific fallback in Python will not usually mimic one in R.
- ▶ Agreement rules out a wide class of bugs; disagreement tells you where to look.

This is the principle behind `/referee2`: a cold-start agent reimplements the analysis independently.

Verification works when errors are imperfectly correlated

No checker is the oracle. The point is to compare independent error processes.

Checker 1	Checker 2	What disagreement reveals
Agent	fresh agent	prompt- or context-specific errors
Agent	human RA	machine vs. human blind spots
R code	Stata/Python code	implementation-specific bugs
Manuscript	source paper	citation and interpretation errors

The standard is zero avoidable errors. One pass is not enough.

Coefficient extraction at scale

I gave many agents a corpus of ~ 600 **APE** papers (Social Catalyst Lab's autonomously generated empirical-policy corpus) and asked them to extract every regression coefficient and standard error.

- ▶ agents read PDFs, located tables, emitted structured rows
- ▶ I built my own t-statistic distribution from the extracted data
- ▶ I planned to run Brodeur-style p-hacking diagnostics on top

Production was the means. The point of doing it was to verify a claim.

The first pass looked like p-hacking

The Brodeur ratio came in around **1.52**. Excess mass piled up at $|t| = 2$.

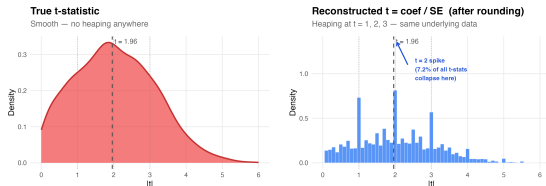
Initial reading: AI-generated empirical papers carry the same publication-bias signature as the human literature.

This was the wrong reading.

Non-random heaping I didn't see

Rounding Turns a Smooth Distribution into a Heaped One

Both panels show the SAME 3,500 simulated regressions. Left: the true $|t|$ -statistic. Right: what you recover when you extract rounded coefficients and SEs from a table and compute coef/SE yourself. 229 of 3187 t -stats (7.2%) collapse to exactly $t = 2.0$ — a pure rounding artifact.



No p-hacking in this simulation. True t -stats drawn from a smooth distribution with no mass point anywhere. Rounding to 1–3 decimal places (mix matching real APE papers) creates the spikes at integer ratios.

Coefficients and SEs in published papers are rounded. Dividing a rounded coefficient by a rounded SE piles t up at integer ratios: 1, 2, 3.

I simulated 3,500 regressions *with no p-hacking*. Same heaping.

Removing 68 exact- $t = 2$ rows dropped the Brodeur ratio from 1.52 to **1.02**. Brodeur et al. have since acknowledged the issue.

Jason Fletcher and Gorkem Ozer caught it. I did not.

This is the experience that led me to write /blindspot — I had started to wonder if I was going to miss the obvious.

Even zero discretion leaves implementation risk

Independent Hallucination Errors. A second experiment.

- ▶ Replicate Dias & Fontes (2024, *AEJ: Economic Policy*) on the same CAPS data.
- ▶ **96 zero-discretion agents.** Every decision prescribed in advance.
- ▶ Vary only what the principal might otherwise vary silently: the package and the covariate set.

Six implementations of the same estimator. Sixteen specifications. Same data, same method, same controls. They should agree.

Six packages implement the same estimator differently

Package	Language	Author
did	R	Callaway & Sant'Anna
ddml	R	Ahrens, Hansen, Schaffer, Wiemann
csdid	Stata	Rios-Avila
csdid2	Stata	Rios-Avila
differences	Python	Dionisi
diff-diff	Python	Gerber

All using doubly robust + not-yet-treated controls + universal base period.

Identical specification across all six.

With no covariates, all six packages agree exactly

did (R)
0.311

ddml (R)
0.311

csdid (Stata)
0.311

csdid2 (Stata)
0.311

differences (Py)
0.311

diff-diff (Py)
0.311

ATT ≈ 0.311
no covariates

The estimator itself is implemented correctly.

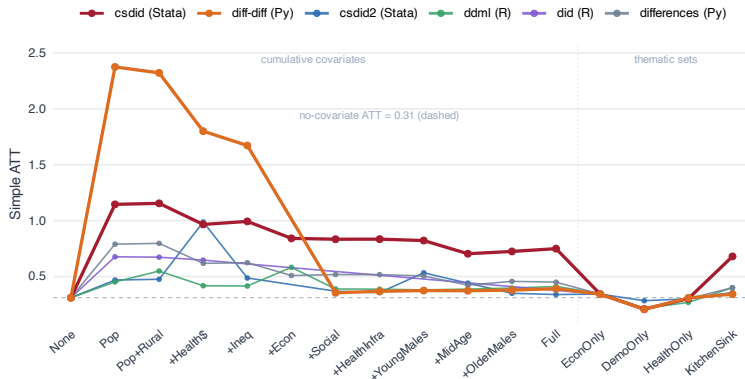
Add one covariate, and the agreement breaks

One covariate: `poptotaltrend` — **municipal population** $\times$ **year**.

Package	ATT
ddml (R)	0.45
csdid2 (Stata)	0.45
did (R)	0.53
differences (Python)	0.67
csdid (Stata)	1.15
diff-diff (Python)	2.38

Same data. Same estimator. Same single covariate. A 5 $\times$ spread.

The disagreement fans out as covariates accumulate



Each line is one of six packages, plotted across 16 specifications. All start at the same point and then fan out.

The smoking gun: remove that one variable

Run specs using only economics, demographics, or health covariates, excluding `poptotaltrend`.

All six packages agree to within 0.007.

One variable caused the entire 5× spread.

The matrix inversion is the easy part

The propensity-score logit's design matrix X has a column `popttotaltrend` (population $\times$ year). For a 50,000-person municipality in 2010, that single cell is **100,500,000**. The variable lives on the order of 10^{10} .

The other covariates live on the order of 0.01 to 16,000. Mixed in the same X , the condition number of $X'X$ explodes to roughly 3.6×10^9 .

Below machine epsilon (2.22×10^{-16}): the computer literally cannot distinguish the columns at 64-bit precision. Inverting $X'X$ — which every propensity-score logit needs — produces garbage.

But this is just a logistic regression. If $X'X$ is hard to invert, shouldn't *all six* packages have a hard time with it?

The answer turns out to be no.

Six “logistic regressions” are not the same logistic regression

Package	Lang	Routine called	Optimizer
did	R	<code>glm(probit)</code>	IRLS
ddml	R	<code>glmnet Lasso (DR) + glm(probit) trim</code>	L1 coord. descent + IRLS
csdid	Stata	<code>probit, iterate(1000) difficult + drimp</code>	Newton–Raphson
csdid2	Stata	<code>probit, difficult ltol(0) tol(1e-5)</code>	Newton–R., relaxed tol.
differences	Py	<code>statsmodels.Probit().fit()</code>	statsmodels Newton
diff-diff	Py	<code>statsmodels.Probit() + CallawaySantAnna</code>	statsmodels + internal

Same name on the API. Different optimizer. Different singular-matrix behavior. The user does not see this.

Each row is the actual call inside the package, recovered from the source.

Is this wrong?

What we found:

- ▶ `did silently` returns all-zero ATTs without raising
- ▶ `csdid` requires `iterate(1000)` difficult to nominally converge
- ▶ `csdid2` runs with relaxed tolerance: “stabilized but not formally converged”
- ▶ `ddml`’s Lasso auto-standardizes inside the learner
- ▶ `statsmodels` converges silently — possibly to wrong values

Honest answer: no individual choice is indefensible. Each package author made a reasonable engineering call. But the choice is invisible to the user. The documentation says “logistic regression.”

The new researcher degrees of freedom: sort into language → package → hidden algorithmic action. Three layers; only the first appears in methods.

The fix: z-score the covariates, or divide `poptotaltrend` by 10^6 . The ill-conditioning is the trigger; remove the trigger and most of the divergence collapses.

Hidden actions become detectable only when independent agents replicate one another.

Other issues found

1. Cohort-specific propensity scores. CS fits a separate propensity-score logit *per treatment cohort*. With 500 treated municipalities split across cohorts, each PS regression sees only N_c observations. If k is too large for that N_c , you get erratic perfect separation — wrapped inside the logistic, independent of the algorithmic heterogeneity above.

2. Silent covariate dropping. Some logistics, when they cannot estimate the full X , silently drop the covariates and regress D on a constant only — without telling the user. The package hides the propensity-score coefficient output and often does not store the propensity scores after estimation. The researcher — and especially Claude in automation mode — may not realize what just happened.

Hidden actions become detectable only when independent agents replicate one another.

Note: `diff-diff` (Gerber) has since been updated; the Round-4-era limitation has been addressed.

Agents make expensive verification routines cheap enough to run

AI did not magically know the answer.

The workflow made the discrepancy visible:

- ▶ many independent agents
- ▶ multiple languages
- ▶ six packages
- ▶ forced comparison to a common target

Agents are useful because they make expensive verification routines cheap enough to run.

5

What To Do Monday Afternoon

Start with audits whose outputs you can judge

Start with audits whose outputs you can inspect:

1. **Citation audit:** Does my literature review accurately represent each cited paper?
2. **Repo audit:** Can a fresh agent install, run, and map this replication package?
3. **Code audit:** Can a fresh agent reimplement my analysis in another language?
4. **Referee audit:** What would a skeptical reader attack first?
5. **Deck audit:** Does my presentation make one clear claim per slide?

Do not begin with the task you least understand. Begin where you can judge the output.

A good assignment makes the output inspectable

Five assignment rules:

1. Give it a folder with the relevant files.
2. State the role: producer, auditor, adversary, replicator.
3. Make the output concrete: table, memo, deck, diff, error list.
4. Force source-grounding: quote line numbers, cite files, compare outputs.
5. Use fresh agents for verification.

The unit of progress is not a clever prompt. It is a repeatable workflow.

Homework: build a skill, do not just borrow one

If you make only one skill after this talk, make it a citation-audit skill. Pick a repeating task and turn it into a procedure your agent can run.

Why build instead of borrow. Ethan Mollick has noted that open source for skills is not the same as open source for code. A skill carries the author's prompts, assumptions, and failure modes. The audit surface is fuzzier than source. You inherit what you install.

Why I show you mine anyway. The point is the pattern, not the prompt. Fork, rewrite, replace — the right thing to copy is the design move, not the trust.

Help me create a citation-audit skill for my papers.

It should: identify each cited claim;

spawn one agent per cited paper; compare my claim to source;

return accurate / overstated / missing / unclear.

Ask me questions and iterate before writing the final skill.

Budget for the real tool, not the demo tier

Agentic research work is not the same as asking a chatbot one question.

For long-running tasks, you want the strongest available model and enough usage headroom:

- ▶ Agents read many files, write code, run it, hit errors, inspect output, and try again.
- ▶ Smaller or capped models may look fine in demos but time out once you depend on the workflow.
- ▶ The relevant comparison is not free vs. paid. It is whether the agent can finish the job.

As of this talk: **Codex is roughly half the monthly price of Claude Max** — about \$100 vs. \$200.

If agents become part of your research process, buy enough model to avoid making the workflow brittle.

When agents fail, ask where the workflow failed

When an LLM underdelivers, there are two possible explanations:

Model failure

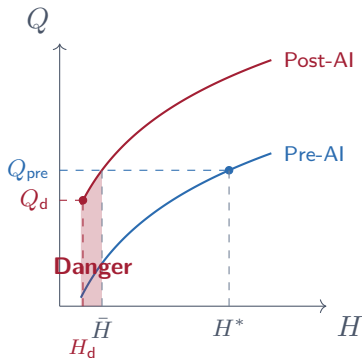
The model genuinely lacks the capability, knowledge, or reasoning required.

Workflow failure

The user assigned the wrong task, gave the wrong context, allowed too much discretion, or failed to verify.

In my experience, workflow failure is usually the first suspect.

Deskilling is also a verification risk



AI can reduce the time and attention used in production.

That is the gain. But skill often came from exactly that time and attention.

- ▶ less production practice can mean less verification skill
- ▶ verification depends on accumulated judgment
- ▶ replication costs are falling toward zero
- ▶ outside scrutiny and penalties may rise

Be cognizant of the deskilling margin.

The optimistic path is more verification per unit of attention

Deskilling is possible. Lost attention is possible.

But the alternative is also possible:

- ▶ more citation claims checked against sources
- ▶ more replication packages actually run
- ▶ more code implemented twice
- ▶ more papers exposed to skeptical readers before submission
- ▶ more talks shaped around audience, not slide labor

The optimistic path is not less human judgment. It is more verification per unit of human attention.

The new skill is knowing what must be verified

The scarce skill is not typing prompts.

The scarce skill is knowing:

- ▶ what to delegate
- ▶ what to constrain
- ▶ what to audit independently
- ▶ what disagreement means
- ▶ when the final judgment must be human

AI agents make production cheap. The researcher's job is to make verification systematic.

Setup links are now an attack surface

Agent workflows often ask you to copy commands into a terminal.

That changes phishing:

- ▶ cloned sites can imitate `claude.ai`, GitHub, or package docs
- ▶ a copy button can hide a dangerous install command
- ▶ agents can browse, compare domains, inspect repos, and explain commands
- ▶ do not paste terminal commands you do not understand

Use agents defensively too: ask them to verify the source before you trust the setup.

Recent threat reports: Google Threat Intelligence on AI-assisted social engineering; Cloud Security Alliance on indirect prompt injection.

Biases from Cues and Nudges

Can context quietly steer an agent toward a biased estimator?

An experiment on the research process itself

Cheap agents let me treat the agent as the experimental unit.

- ▶ Many agents receive the same data and the same task.
- ▶ I vary only the context around them, not the question.
- ▶ The outcome is behavioral: which econometrics does each agent choose?

The question is not whether an agent is right. It is whether the context moves what the agent does.

The data: U.S. state minimum wages, 1990 to 2022

- ▶ A state-year panel of 51 states across 33 years.
- ▶ Outcome: the teen employment-to-population ratio.
- ▶ Treatment: a state's effective minimum wage binding above the federal floor.
- ▶ Variation: states raise their minimums on a staggered schedule, with federal increases layered on top.

Staggered timing is exactly the setting where the choice of estimator starts to matter.

Under staggered timing, TWFE hides a bias term

$$\hat{\beta}^{\text{TWFE}} = \text{VWATT} + \text{VWPT} - \Delta\text{ATT}$$

- ▶ VWATT: a variance-weighted average of the true treatment effects.
- ▶ VWPT: a variance-weighted parallel-trends term.
- ▶ ΔATT : the penalty from using already-treated units as controls.

When effects grow or fade over time, the ΔATT term contaminates the estimate and can flip its sign. Goodman-Bacon (2021).

Why the minimum wage, 1990 to 2022

- ▶ **It is empirically contested.** The literature disagrees, so the training data carries no strong prior for the agent to lean on.
- ▶ **The federal floor rises during the panel.** Federal increases eventually bind everywhere, so there are no clean never-treated states left as controls.

A contested question with no clean controls is where estimator choice does the most damage.

A worst case for TWFE, and therefore a clean test

- ▶ With a binary “the minimum wage rose and binds” indicator on the full panel, TWFE collapses to the sum of two 2×2 comparisons: not-yet-treated and already-treated.
- ▶ The already-treated comparison is precisely the one that biases TWFE under staggering.
- ▶ Callaway–Sant’Anna can identify only binary ATTs between federal minimums; TWFE will accept any representation, binary or continuous dose.

So by construction, TWFE here is the biased estimator.

The ground truth is not a number

- ▶ I am not claiming the true employment effect of the minimum wage is some value.
- ▶ The ground truth is structural: in this design, TWFE is biased and Callaway–Sant’Anna is not.

So the clean behavioral question is whether a cue pushes an agent toward the estimator we already know is biased.

Do agents take hidden, unreported actions in response to cues?

Three cues

- ▶ A JEL-style methodology document.
- ▶ An estimator menu.
- ▶ A preregistration plan.

Two models

- ▶ GPT-5.5 (closed weight).
- ▶ Kimi K2.6 (open weight).

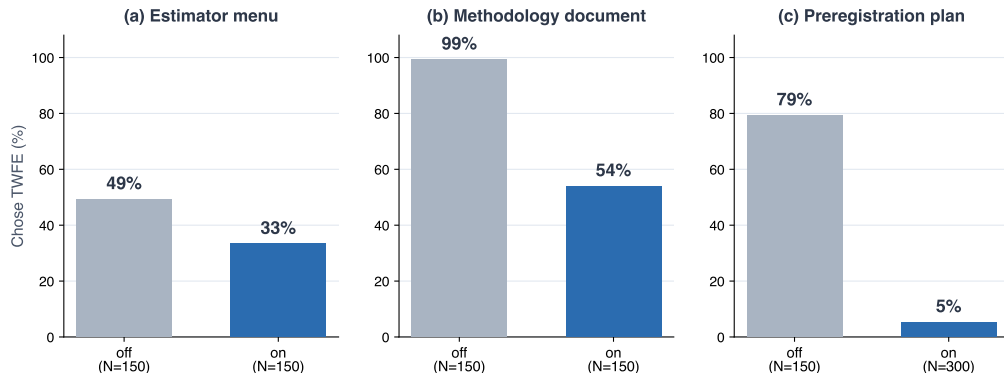
A hidden action is the agent quietly switching estimators in response to context it was never told to obey.

Simple statistics, read across the treatment arms

- ▶ A linear probability model of choosing TWFE on the cue.
- ▶ A regression of the reported ATT estimate on the cue.
- ▶ A Kolmogorov–Smirnov test on the distribution of estimates across arms.

The same logic covers the other hidden actions: opening the preregistration plan, and redefining treatment as a continuous dose. **Nothing fancy. The design does the work, not the estimator.**

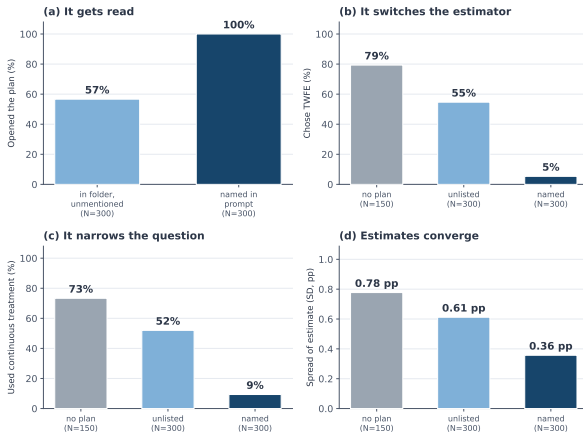
Every cue moves the estimator



GPT-5.5; share choosing two-way fixed effects. Each experiment fixes the other cues and toggles one: (a) no document, menu off ☐ on; (b) no menu, document off ☐ on; (c) document on, plan off ☐ on. The plan 'on' bar pools the two plan wordings (N=300). N under each bar.

GPT-5.5. Share choosing TWFE with each cue off vs on; the other cues are held fixed.

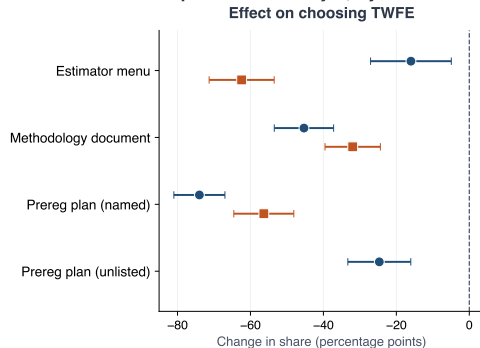
A preregistration plan, read unprompted, reshapes the analysis



GPT-5.5. A preregistration plan placed in the working directory, delivered three ways: no plan / in folder but unmentioned / named in the prompt. (a) share that opened it; (b) share choosing TWFE; (c) share that redefined treatment as continuous rather than the binary ATT; (d) cross-agent spread of the binary ATT (standard deviation, in percentage points — note: not a share). N under each bar.

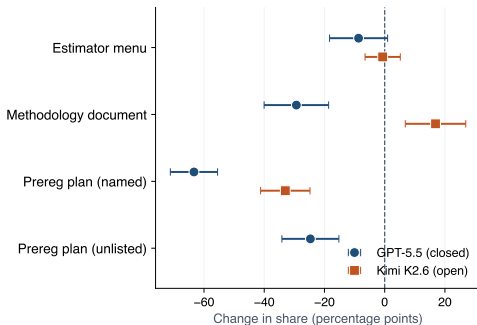
The same pull, in a closed model and an open one

How hard each cue pushes the analyst, by model



LPM of each binary outcome on the cue (treatment vs control), HC3 95% CI; points are percentage-point effects. Negative = the cue moves the agent away from TWFE / away from continuous treatment.

Effect on using continuous treatment



Each point is an LPM of the outcome on the cue, HC3 95% CI. Negative means the cue moves the agent away from TWFE.

Keep your hands on the wheel

- ▶ The agents took these actions on their own, and did not report them.
- ▶ A cue you never considered can change the estimator, the estimand, and the estimate.

You cannot hand off the wheel. You have to understand the analysis, direct it, and verify it.

When agents fail, what failed?

It is too simple to call every bad output a hallucination.

Production failed

The agent made the wrong thing: bad code, bad draft, bad search, bad translation.

Verification failed

The output looked plausible, but nobody checked the right object.

Direction failed

The human gave the wrong task, context, constraints, or stopping rule.

My prior: workflow failures explain more agent failures than people think.

6

Looking Ahead

It once sounded silly to call a machine a substitute for thinking

For two hundred years, machines replaced muscle and then routine. Thinking was ours.

- ▶ In $Q = f(H, M)$, machine time now stands in for human time in cognitive work.
- ▶ That is the flat isoquant from this morning.
- ▶ I think that is where we are, and it is early.

I do not know where this is going. I do not think anyone does.

Are we complements or substitutes in the new production function?

If we are complements

- ▶ The machine makes our time more productive.
- ▶ Demand for our time rises.
- ▶ This was true of the tools we grew up with.

If we are substitutes

- ▶ The machine does the task in our place.
- ▶ Demand for our time falls.
- ▶ What remains for us is whatever the machine cannot do.

The answer need not be the same for every task, or for every person.

And substitution is only half of it: if cheaper output raises demand enough, the output effect can outweigh the substitution effect.

It is an empirical question, and the data are not in.

Computers hollowed out the middle. Will agents hollow out the top?

What David Autor showed about computers

- ▶ Computers took over routine tasks, which were the core of middle-skill jobs.
- ▶ Clerical and production work shrank as a share of employment.
- ▶ Jobs grew at the top and at the bottom, and the middle thinned.

What we cannot yet say about agents

- ▶ The tasks at risk now are analysis, writing, and code.
- ▶ Those belong to the upper middle class and the lower upper class.

Whether that tier hollows out the way the middle did is really unclear.

Autor, Levy, and Murnane (2003); Autor (2015).

The old tools are as useful as they ever were

- ▶ **Production theory.** Isoquants, marginal products, and the elasticity of substitution tell us what to measure.
- ▶ **Labor markets tied to product markets.** The demand for our labor is derived from the demand for what we make.
- ▶ **The circular flow.** Someone's spending is someone else's income, including when the spender is a data center.

The world is more complicated than it was two years ago. The economics we already have is still how to think about it.

We have been here before: the first machines made the weavers rich

Cotton spinning was mechanized first.

Hours of labor to spin 100 pounds of cotton

By hand, early 1700s	more than 50,000
Crompton's mule, 1780	2,000
Power-assisted mules, about 1795	300

- ▶ Yarn became cheap, and cheap yarn needed weavers.
- ▶ In its golden age, from about 1780 to 1800, a weaver worked four days a week and earned 40 shillings.

The first machine was a complement. That is why economists assumed machinery was painless.

Acemoglu and Johnson (2024), drawing on Chapman (1987).

Then the machines came for weaving

Year	Power looms in Britain
1813	2,400
1820	14,150
1833	at least 100,000

- ▶ At first the power loom was hardly faster than a handloom.
- ▶ By the mid-1820s, one boy tending two looms could match fifteen cottage weavers.
- ▶ Spinning was already mechanized, so there was no neighboring trade to absorb them.

The same industry, one generation later, and this time the machine was a substitute.

Acemoglu and Johnson (2024), drawing on Baines (1835) and Landes (2003).

The Luddites were skilled workers defending a trade

- ▶ They began in Nottinghamshire in March 1811, among the framework knitters.
- ▶ In 1812 the movement spread to the cloth finishers of Yorkshire and the cotton weavers of Lancashire.
- ▶ They signed their letters “Ned Ludd,” a man who never existed.
- ▶ They broke particular machines: the ones being used to cut pay and to replace trained hands.

They were not afraid of technology. They were bargaining over who would bear its cost.

Thompson (1963); Hobsbawm (1952).

The government answered with soldiers and the gallows

- ▶ In 1812 Parliament made breaking a machine punishable by death.
- ▶ About 12,000 soldiers were sent to the textile districts.
- ▶ In January 1813, seventeen men were hanged at York.

The workers had no vote, and unions were illegal.

Eric Hobsbawm called machine breaking “collective bargaining by riot.” It was the only bargaining they were allowed.

Thompson (1963); Hobsbawm (1952).

What they lost was autonomy, and not only income

At the handloom

- ▶ The weaver chose when to work and how hard.
- ▶ The family worked together, at home.
- ▶ He sold cloth, and he did not sell his labor.

In the mill

- ▶ The owner set the hours and the pace.
- ▶ The work was watched and it was monotonous.
- ▶ Owners hired women and children, who were cheaper and easier to control.

E. P. Thompson's phrase for the change: a self-motivated man became a "hand."

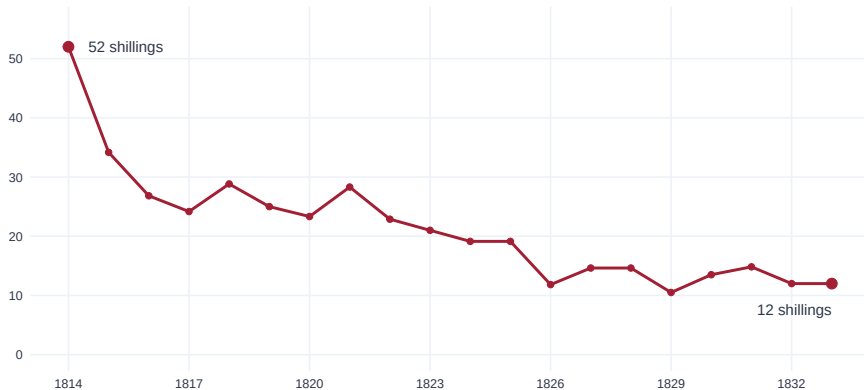
Many weavers took starvation wages at home over higher wages in the mill.

Thompson (1963) and Hobsbawm (1999), as discussed in Acemoglu and Johnson (2024).

First the pay fell

A handloom weaver's family lost three quarters of its earnings in twenty years

Weekly earnings of a family of six near Colne, Lancashire, in nominal shillings

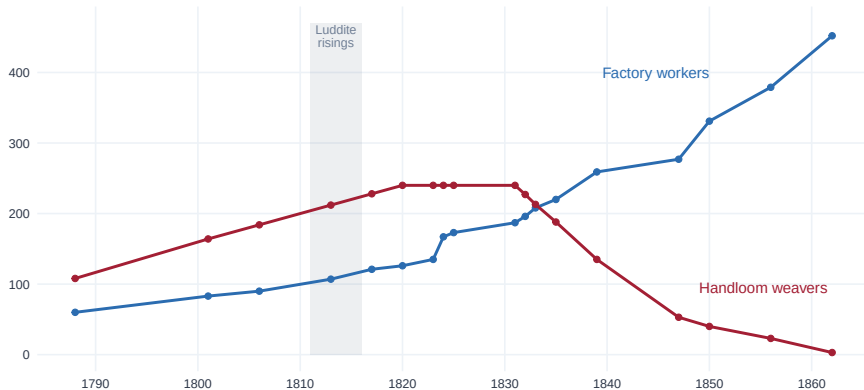


Data: Wood (1910), as reported in Acemoglu and Johnson (2024), Table 1.

Only later did the jobs go

The handloom weavers held on for a generation, and then they were gone

Employment in the British cotton industry, thousands of workers



Data: Wood (1910), as reported in Acemoglu and Johnson (2024), Table 2.

Wages adjusted for twenty years before employment did

- ▶ A weaving family's earnings fell from 52 shillings a week to 12 between 1814 and 1833.
- ▶ Over those same years the number of handloom weavers did not fall. It stayed at its peak of 240,000.
- ▶ They worked longer for less: by the 1830s, fourteen to sixteen hours a day, six days a week.
- ▶ Their real wage fell to about a quarter of what it had been.
- ▶ The jobs disappeared afterward, from 240,000 in 1831 to 3,000 in 1862.

If you look only at employment, you miss the first twenty years of the story.

Acemoglu and Johnson (2024), Tables 1 and 2, from Wood (1910).

David Ricardo changed his mind

-
- 1817** In the first edition of his *Principles*, machinery does workers no harm.
 - 1819** He tells Parliament that “machinery did not lessen the demand for labor.”
 - 1819** Peterloo: a reform meeting of perhaps 60,000 in Manchester is broken up by force.
 - 1821** The third edition adds a chapter, “On Machinery.”
-

In that chapter he wrote that his opinions had “undergone a considerable change.”

The best economist of his day got it wrong in real time, saw the evidence, and said so in print.

Acemoglu and Johnson (2024).

Shared gains came later, from new tasks and from worker voice

- ▶ Real wages across the British economy were roughly flat until the 1830s.
- ▶ The average wage in cotton stayed below its 1806 level through 1862.
- ▶ By many estimates, the recovery took longer than a working lifetime.

What changed after about 1850

- ▶ Railways and heavy industry created tasks that had not existed.
- ▶ The vote widened, and unions won the right to bargain.

Acemoglu and Johnson's reading: none of it was preordained. "These were choices."

Acemoglu and Johnson (2024).

Research is an Anna Karenina problem

“Happy families are all alike; every unhappy family is unhappy in its own way.”

Tolstoy, *Anna Karenina*

- ▶ A happy family needs five or six things, and it needs every one of them.
- ▶ Being excellent at four does not make up for the one that is missing.
- ▶ A mousetrap is the same: take away any one part and it catches nothing.

Good research is like a happy family. It succeeds one way, with everything present, and it can fail in many ways.

Shockley had a production function for science

In 1957, William Shockley wrote “On the Statistics of Individual Variations of Productivity in Research Laboratories.”

He argued that research productivity has a long-tailed, log-normal shape: a small number of workers produce enormously more measurable output.

$$P = F_1 F_2 F_3 F_4 F_5 F_6 F_7 F_8 \quad \Rightarrow \quad \log P = \sum_{j=1}^8 \log F_j$$

He did not call it a production function, but that is what it is. In our language it is Cobb–Douglas: set any one input to zero and output is zero.

Shockley's inputs mix production and verification

In 1957, these were bundled inside the research worker. With agents, they begin to separate.

Shockley's factor	Task		Agent pressure
Think of a good problem	B	↑	search, recombination
Work on it	P	↑	tireless execution
Recognize a worthwhile result	B	↓	if attention is outsourced
Know when to stop and write	B	↔	still judgment
Write adequately	P	↑	drafting, editing
Profit from criticism	B	↑	referee simulation
Submit to a journal	B	↔	incentives remain human
Persist through revisions	B	↑	cheap iteration

P = mostly production, B = bundled production and verification

The new skill is harnessing both sides through trial, error, and experimentation.

Because it multiplies, your smallest input caps your output

Output is the *product* of eight inputs, not their sum:

$$P = F_1 F_2 F_3 F_4 F_5 F_6 F_7 F_8$$

Find a problem, work on it, recognize a result, know when to stop, write it up, profit from criticism, submit it, persist through revision.

His own illustration: a worker 50% better on *each* of the eight factors is about **25×** as productive ($1.5^8 \approx 25$).

The binding input is different for each of us. That is every unhappy family, unhappy in its own way.

The eight inputs have not changed. Who supplies them has.

- ▶ Agents can now supply several of Shockley's inputs, and supply them cheaply.
- ▶ Every one of the eight still has to be present.
- ▶ If an agent does the work and nobody recognizes whether the result is worthwhile, the product is still zero.

Learning to ride this new bull is going to be a challenge, and each of us will fall off in a different place.

But I think Shockley was right. The inputs to good research in 2026 are the inputs he listed in 1957.

What I am sure of: the human in the loop is essential

I do not know whether we are complements or substitutes, or who will be hollowed out.

What my own research on agents doing causal inference tells me

- ▶ Agents take actions you did not ask for.
- ▶ The finished output does not show that they did.
- ▶ Those hidden actions can change the answer.

So someone who understands the work has to check it, every time.

Whether people know that yet is another matter. I am convinced of it.

**Production is cheap.
Verification is the craft.**

Scott Cunningham

Baylor University · causalinf.substack.com